

SR3 六自由度机械臂课程设计

项目二：正运动学、逆运动学与雅可比

项目三：路径规划、轨迹生成与跟踪控制

从零开始的超详细原理、推导、代码与测试教程

课程名称： 机器人课程设计

机械臂型号： xMate SR3

小组成员： _____

学号： _____

指导教师： _____

完成日期： 2026 年 6 月 14 日

说明： 本稿既可作为课程设计报告底稿，也可作为答辩复习讲义。

写在最前面：这份文档怎么读

这份文档故意写得非常慢、非常细。阅读时不要求提前懂矩阵、机器人学或 MATLAB。每遇到一个公式，文档都会尽量回答四个问题：

1. 这个量是什么？
2. 这个公式从哪里来？
3. 它在机器人运动中表示什么？
4. 它在代码中为什么要这样写？

本报告严格对应当前仓库中的 SR3 实现。需要提前说明一个事实：

`inverse_kinematics_analytical.m` 虽然沿用了“解析逆运动学”的文件名，但当前代码实际采用的是多初值阻尼最小二乘数值搜索，再通过正运动学反校验和去重得到多组候选解。因此本文会分开讲：

- 课程题目要求的 Pieper 分离法和闭式解析逆解思想；
- 当前仓库真正运行的 DLS 数值逆解算法；
- 两者为什么不同，以及报告和答辩时应如何如实说明。

目录

写在最前面：这份文档怎么读	i
第一章 零基础预备知识	1
1.1 机器人运动学到底在研究什么	1
1.2 标量、向量和矩阵	1
1.2.1 标量	1
1.2.2 向量	1
1.2.3 矩阵	2
1.3 坐标系为什么必不可少	2
1.4 三维旋转	2
1.4.1 绕三个基本轴旋转	2
1.4.2 旋转矩阵的三个重要性质	3
1.5 齐次变换矩阵	3
1.6 标准 DH 与改进 DH	3
1.7 关节角偏置 offset	4
1.8 叉乘及其物理意义	4
1.9 MATLAB 最低限度语法	5
第二章 项目二：正运动学、逆运动学与雅可比	6
2.1 项目二要解决的完整问题	6
2.2 SR3 的 MDH 参数	6
2.3 正运动学如何一步一步计算	7
2.3.1 第一步：把关节角变成 MDH 表	7
2.3.2 第二步：每一行变成一个局部变换	7
2.3.3 第三步：把六步串起来	7
2.4 逆运动学为什么比正运动学难	7

2.5	课程要求的 Pieper 分离法	7
2.5.1	核心思想	7
2.5.2	腕中心	8
2.5.3	前三关节的几何解	8
2.5.4	后三关节的姿态解	9
2.5.5	腕部奇异	9
2.6	当前代码真正使用的 DLS 数值逆运动学	9
2.6.1	把逆运动学改写成“不断纠错”	9
2.6.2	小运动近似与雅可比	9
2.6.3	阻尼最小二乘	10
2.6.4	阻尼系数的影响	10
2.7	几何雅可比如何推导	10
2.7.1	雅可比的尺寸	10
2.7.2	转动关节的一列	11
2.7.3	MDH 中为什么不能直接拿累计矩阵的 z 轴	11
2.8	姿态误差的轴角表示	11
2.9	多初值为什么能找到多组解	11
2.10	角度归一化与解的选择	12
2.11	当前实现的优点与局限	12
2.11.1	优点	12
2.11.2	局限	12
第三章	项目三：路径规划、轨迹生成与跟踪	13
3.1	路径、轨迹、控制不是同一个东西	13
3.2	为什么选择关节空间规划	13
3.3	场景数据结构	13
3.4	碰撞检测：为什么不能只看末端	14
3.4.1	点到线段最短距离	14
3.4.2	状态合法不等于边合法	14
3.5	先试直线为什么合理	14
3.6	RRT 的基本思想	15
3.6.1	最近邻	15
3.6.2	Steer	15

3.7 为什么使用双向 RRT-Connect	15
3.8 树的数据结构和路径回溯	16
3.9 目标偏置与确定性	16
3.10 当前关键参数为什么这样写	17
3.11 Shortcut 路径简化	17
3.12 手工绕行兜底	17
3.13 路径弧长参数化	17
3.14 五次多项式时间缩放从哪里来	18
3.15 从标量进度得到每个关节的轨迹	18
3.16 PD 控制器	19
3.17 Euler 积分	19
3.18 项目三的局限与改进方向	19
第四章 代码阅读、测试判据与答辩准备	21
4.1 项目二逐文件阅读顺序	21
4.2 项目二核心代码逐句解释	21
4.2.1 正运动学累计	21
4.2.2 MDH 关节轴	21
4.2.3 DLS 更新	22
4.3 项目三逐文件阅读顺序	22
4.4 项目三核心代码逐句解释	22
4.4.1 直线优先	22
4.4.2 扩展一步	23
4.4.3 轨迹时间缩放	23
4.5 测试到底在检查什么	23
4.5.1 IK 自洽测试	23
4.5.2 雅可比中心差分测试	23
4.5.3 路径测试	24
4.5.4 轨迹测试	24
4.5.5 跟踪测试	24
4.6 推荐的实验结果图	24
4.7 最小安全距离如何计算	25
4.8 常见报错与定位	25

4.9 答辩可能提问与简洁回答	25
4.10 AI 工具使用声明模板	26
附录 A 核心 MATLAB 源码附录	27
A.1 项目二核心代码	27
A.1.1 机器人参数	27
A.1.2 MDH 参数表构造	28
A.1.3 单节 MDH 变换	29
A.1.4 正运动学	30
A.1.5 几何雅可比	31
A.1.6 DLS 数值逆运动学	34
A.1.7 多初值逆运动学封装	36
A.1.8 逆解选择	39
A.2 项目三核心代码	40
A.2.1 路径规划	40
A.2.2 轨迹生成	48
A.2.3 PD 控制器	50
A.2.4 跟踪仿真	51
附录 B 运行命令与提交前检查表	56
B.1 MATLAB 运行方式	56
B.2 LaTeX 编译方式	56
B.3 提交前检查表	56
结语	58

插图

3.1 当前路径规划器的总体流程	16
----------------------------	----

表格

2.1 当前代码采用的 SR3 MDH 参数，长度单位为 mm	6
---	---

第一章 零基础预备知识

1.1 机器人运动学到底在研究什么

想象桌面上放着一条由六个旋转关节连接起来的机械臂。每个关节都像门轴一样可以转动。六个关节角写成

$$\mathbf{q} = [q_1, q_2, q_3, q_4, q_5, q_6].$$

这里的 q_i 不是坐标点，而是第 i 个电机轴转过的角度。当前项目统一使用 **弧度**。常见换算为

$$180^\circ = \pi \text{ rad}, \quad 1^\circ = \frac{\pi}{180} \text{ rad}.$$

机器人学里最常见的两个问题是：

正运动学（FK） 已知六个关节角，计算机械臂末端的位置和朝向。

逆运动学（IK） 已知希望末端到达的位置和朝向，反过来求六个关节角。

正运动学像“知道每节手臂怎么弯，算手掌在哪里”；逆运动学像“我想让手掌碰到杯子，反推肩、肘、腕应该怎么转”。

1.2 标量、向量和矩阵

1.2.1 标量

只有一个数的量称为标量，例如球形障碍物的半径 $r = 90 \text{ mm}$ 。

1.2.2 向量

三个数排成一行可以表示三维空间中的位置：

$$\mathbf{p} = \begin{bmatrix} x \\ y \\ z \end{bmatrix}.$$

例如 $\mathbf{p} = [480, 200, 90]^T$ 表示相对于基坐标系，沿 x 方向 480 mm、沿 y 方向 200 mm、沿 z 方向 90 mm。上标 T 表示转置，即把行向量变成列向量。

向量的长度称为二范数：

$$\|\mathbf{p}\|_2 = \sqrt{x^2 + y^2 + z^2}.$$

代码中的 `norm(p)` 就是在算这个量。

1.2.3 矩阵

矩阵可以看成按行列摆放的一组数字。旋转矩阵是一个 3×3 矩阵：

$$\mathbf{R} = \begin{bmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{bmatrix}.$$

它的三列分别是新坐标系的 x 、 y 、 z 轴在旧坐标系中指向哪里。

1.3 坐标系为什么必不可少

“末端在 (100, 200, 300)” 这句话并不完整，因为必须说明相对于谁测量。机器人通常给每根连杆建立一个坐标系：

$$\{0\}, \{1\}, \dots, \{6\}.$$

$\{0\}$ 是固定在机器人底座上的基坐标系， $\{6\}$ 固定在末端。

符号 ${}^0\mathbf{p}_6$ 表示“坐标系 6 的原点，在坐标系 0 中的位置”。符号 ${}^0\mathbf{R}_6$ 表示“坐标系 6 相对于坐标系 0 的朝向”。

1.4 三维旋转

1.4.1 绕三个基本轴旋转

绕 x 轴旋转 α ：

$$\mathbf{R}_x(\alpha) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \alpha & -\sin \alpha \\ 0 & \sin \alpha & \cos \alpha \end{bmatrix}.$$

绕 y 轴旋转 β ：

$$\mathbf{R}_y(\beta) = \begin{bmatrix} \cos \beta & 0 & \sin \beta \\ 0 & 1 & 0 \\ -\sin \beta & 0 & \cos \beta \end{bmatrix}.$$

绕 z 轴旋转 θ :

$$\mathbf{R}_z(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

矩阵乘法的顺序不能交换

通常 $\mathbf{R}_x \mathbf{R}_z \neq \mathbf{R}_z \mathbf{R}_x$ 。先绕 x 转再绕 z 转，与先绕 z 再绕 x 转，最终朝向一般不同。因此 MDH 公式中的乘法顺序必须与代码完全一致。

1.4.2 旋转矩阵的三个重要性质

合法旋转矩阵满足

$$\mathbf{R}^T \mathbf{R} = \mathbf{I}, \quad \mathbf{R}^{-1} = \mathbf{R}^T, \quad \det(\mathbf{R}) = 1.$$

第二条非常重要：旋转的逆可以直接用转置得到。项目二中

$$\mathbf{R}_{36} = \mathbf{R}_{03}^T \mathbf{R}_{06}$$

正是利用了这一性质。

1.5 齐次变换矩阵

只用 $\mathbf{R}$ 能描述朝向，只用 $\mathbf{p}$ 能描述位置。为了把二者放在一个矩阵里，机器人学使用 4×4 齐次变换矩阵：

$$\mathbf{T} = \begin{bmatrix} \mathbf{R} & \mathbf{p} \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

若

$${}^0\mathbf{T}_1 = \begin{bmatrix} {}^0\mathbf{R}_1 & {}^0\mathbf{p}_1 \\ 0 & 1 \end{bmatrix}, \quad {}^1\mathbf{T}_2 = \begin{bmatrix} {}^1\mathbf{R}_2 & {}^1\mathbf{p}_2 \\ 0 & 1 \end{bmatrix},$$

那么从 0 到 2 的变换为

$${}^0\mathbf{T}_2 = {}^0\mathbf{T}_1 {}^1\mathbf{T}_2.$$

这就是正运动学循环中不断执行 $\mathbf{T} = \mathbf{T} * \mathbf{A}_i$ 的原因。

1.6 标准 DH 与改进 DH

DH 方法用四个参数描述相邻两根连杆：

$$a, \quad \alpha, \quad d, \quad \theta.$$

直观解释如下：

参数	含义
a	沿某个 x 轴方向的连杆长度
α	相邻两个关节轴之间的扭转角
d	沿关节轴方向的偏移距离
θ	绕关节轴旋转的角度；转动关节中它含有关节变量

本项目采用改进 DH (MDH)：

$${}^{i-1}\mathbf{T}_i = \mathbf{R}_x(\alpha_{i-1})\mathbf{T}_x(a_{i-1})\mathbf{R}_z(\theta_i)\mathbf{T}_z(d_i).$$

把四个基本变换相乘，得到代码 `mdh_transform.m` 使用的矩阵：

$$\begin{bmatrix} c_\theta & -s_\theta & 0 & a \\ s_\theta c_\alpha & c_\theta c_\alpha & -s_\alpha & -ds_\alpha \\ s_\theta s_\alpha & c_\theta s_\alpha & c_\alpha & dc_\alpha \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

其中 $c_\theta = \cos \theta$, $s_\theta = \sin \theta$, 其余同理。

1.7 关节角偏置 offset

真实电机零位不一定等于数学模型中 $\theta_i = 0$ 的姿态，所以代码使用

$$\theta_i = q_i + \text{offset}_i.$$

SR3 的偏置为

$$\text{offset} = \left[\pi, \frac{\pi}{2}, \frac{\pi}{2}, \pi, 0, 0 \right].$$

例如电机读数 $q_2 = 0$ 时，MDH 公式实际使用 $\theta_2 = \pi/2$ 。漏掉偏置会让整台机器人模型“从第一步就站错姿势”。

1.8 叉乘及其物理意义

两个三维向量的叉乘为

$$\mathbf{a} \times \mathbf{b} = \begin{bmatrix} a_y b_z - a_z b_y \\ a_z b_x - a_x b_z \\ a_x b_y - a_y b_x \end{bmatrix}.$$

其方向同时垂直于 $\mathbf{a}$ 和 $\mathbf{b}$ 。

对转动关节，如果关节轴方向为 $\mathbf{z}$ ，从轴上的一点到末端的向量为 $\mathbf{p}_n - \mathbf{p}_i$ ，则关节转动产生的瞬时末端线速度方向为

$$\mathbf{z} \times (\mathbf{p}_n - \mathbf{p}_i).$$

这就是几何雅可比线速度部分的来源。

1.9 MATLAB 最低限度语法

代码	含义
<code>q(:)'</code>	先把数据拉成列向量，再转成行向量，保证形状为 $1 \times n$
<code>zeros(6,n)</code>	创建 $6 \times n$ 的全零矩阵
<code>eye(4)</code>	创建 4×4 单位矩阵
<code>A*B</code>	矩阵乘法
<code>A.*B</code>	对应元素逐个相乘
<code>A\b</code>	解线性方程 $Ax = b$ ，通常比 <code>inv(A)*b</code> 稳定
<code>T(1:3,4)</code>	取齐次矩阵前三行、第四列，即位置向量
<code>T(1:3,1:3)</code>	取左上角旋转矩阵
<code>cross(a,b)</code>	三维叉乘
<code>norm(x)</code>	向量长度；矩阵时需留意所用范数
<code>size(A,1)</code>	矩阵行数
<code>end+1</code>	在数组末尾添加新元素
<code>struct(...)</code>	创建带命名字段的数据结构

读代码的核心方法

看到一行代码时，先写出每个变量的尺寸。例如

$$\mathbf{J} : 6 \times 6, \quad \mathbf{e} : 6 \times 1, \quad \mathbf{J}^T(\mathbf{J}\mathbf{J}^T + \lambda^2 \mathbf{I})^{-1} \mathbf{e} : 6 \times 1.$$

尺寸对得上，代码才可能正确；尺寸对不上，公式一定无法直接运行。

第二章 项目二：正运动学、逆运动学与雅可比

2.1 项目二要解决的完整问题

项目二的输入是目标末端位姿

$$\mathbf{T}_{\text{target}} = \begin{bmatrix} \mathbf{R}_{\text{target}} & \mathbf{p}_{\text{target}} \\ 0 & 1 \end{bmatrix},$$

输出是若干组可能的关节角。它不是一个孤立函数，而是一条验证链：

$$\mathbf{q} \xrightarrow{\text{FK}} \mathbf{T} \xrightarrow{\text{IK}} \mathbf{q}' \xrightarrow{\text{FK}} \mathbf{T}'.$$

只有当 $\mathbf{T}' \approx \mathbf{T}$ 时，逆解才可信。

2.2 SR3 的 MDH 参数

表 2.1: 当前代码采用的 SR3 MDH 参数，长度单位为 mm

i	a_{i-1}	α_{i-1}	d_i	θ_i
1	0	0	344	$q_1 + \pi$
2	0	$\pi/2$	0	$q_2 + \pi/2$
3	290	π	0	$q_3 + \pi/2$
4	0	$\pi/2$	290	$q_4 + \pi$
5	0	$-\pi/2$	0	q_5
6	136	$\pi/2$	103.5	q_6

2.3 正运动学如何一步一步计算

2.3.1 第一步：把关节角变成 MDH 表

`build_mdh_table.m` 对每个关节执行

$$\theta_i = q_i + \text{offset}_i,$$

然后把 $[a_{i-1}, \alpha_{i-1}, d_i, \theta_i]$ 放入第 i 行。

2.3.2 第二步：每一行变成一个局部变换

调用

$$\mathbf{A}_i = \text{mdh_transform}(a_{i-1}, \alpha_{i-1}, d_i, \theta_i).$$

$\mathbf{A}_i$ 只描述“从前一坐标系走到当前坐标系”。

2.3.3 第三步：把六步串起来

$${}^0T_6 = {}^0A_1 {}^1A_2 {}^2A_3 {}^3A_4 {}^4A_5 {}^5A_6.$$

代码从 $T = \text{eye}(4)$ 开始，每次右乘一个 $\mathbf{A}_i$ 。同时保存中间结果 $T_all(:, :, i+1)$ ，因为碰撞检测和雅可比不仅需要末端，还需要每个关节原点的位置。

2.4 逆运动学为什么比正运动学难

正运动学只需按确定顺序代入并相乘，通常只有一个结果。逆运动学困难在于：

- 同一末端位姿可能对应多组关节角；
- 有些目标超出工作空间，根本无解；
- 接近奇异位形时，某些关节运动效果重合；
- 三角函数是周期函数， q 与 $q + 2k\pi$ 表示相同转角；
- 机械臂还受关节限位约束。

2.5 课程要求的 Pieper 分离法

2.5.1 核心思想

对后三根关节轴交于一点的六轴球腕机械臂，可以把问题拆开：

1. 前三轴负责把腕中心送到正确位置；
2. 后三轴负责把末端转到正确朝向。

因此先解位置，再解姿态。这称为位置与姿态解耦。

2.5.2 腕中心

在最简单的球腕模型中，末端原点沿末端 z 轴距离腕中心 d_6 ，因此

$$\mathbf{p}_w = \mathbf{p}_6 - d_6 \mathbf{R}_6(:, 3).$$

$\mathbf{R}_6(:, 3)$ 是末端 z 轴在基坐标系中的方向。

当前 SR3 参数并非最简单模板

当前模型第六行同时含 $a_5 = 136$ 和 $d_6 = 103.5$ 。这意味着末端固定偏置不只沿 z_6 方向，简单公式 $\mathbf{p}_w = \mathbf{p} - d_6 \mathbf{R}(:, 3)$ 不足以完整剥离偏置。这也是当前仓库最终转向通用数值 IK 的重要原因。不能把模板球腕公式不加检查地宣称为当前代码的精确实现。

2.5.3 前三关节的几何解

对于理想化平面二连杆，设腕中心为 $[x_w, y_w, z_w]^T$ 。第一轴先决定腕中心位于机器人哪一侧：

$$\theta_1 = \text{atan2}(y_w, x_w).$$

另一肩部构型常可写为 $\theta_1 + \pi$ 。

把三维问题投影到关节 2、3 所在平面：

$$r = x_w \cos \theta_1 + y_w \sin \theta_1, \quad s = z_w - d_1.$$

若两段有效长度为 L_1, L_2 ，根据余弦定理：

$$D = \frac{r^2 + s^2 - L_1^2 - L_2^2}{2L_1L_2}.$$

当 $|D| > 1$ 时三角形无法组成，目标不可达。当 $|D| \leq 1$ 时：

$$\theta_3 = \text{atan2}(\pm \sqrt{1 - D^2}, D).$$

正负号产生肘上、肘下两组解。再由

$$\theta_2 = \text{atan2}(s, r) - \text{atan2}(L_2 \sin \theta_3, L_1 + L_2 \cos \theta_3)$$

求得肩关节俯仰角。

2.5.4 后三关节的姿态解

前三关节确定后，先算 R_{03} ，目标姿态为 R_{06} ，于是

$$R_{36} = R_{03}^{-1} R_{06} = R_{03}^T R_{06}.$$

再把 R_{36} 与腕部三个旋转矩阵的乘积逐元素比较，就能通过 `atan2` 提取 $\theta_4, \theta_5, \theta_6$ 。

一般情况下 θ_5 的正负分支对应腕部翻转，从而形成两组腕部解。于是最多有

$$2(\text{肩}) \times 2(\text{肘}) \times 2(\text{腕}) = 8$$

组解。

2.5.5 腕部奇异

当 $\sin \theta_5 \approx 0$ 时，第 4 轴和第 6 轴的效果耦合。此时不是没有解，而是 θ_4 与 θ_6 各自不唯一，通常只有两者之和或差能够确定。程序必须人为选一个约定，例如令 $\theta_4 = 0$ ，再计算 θ_6 。

2.6 当前代码真正使用的 DLS 数值逆运动学

2.6.1 把逆运动学改写成“不断纠错”

假设当前猜测为 q_k 。用正运动学得到当前位姿 $T(q_k)$ ，与目标比较得到六维误差：

$$e = \begin{bmatrix} e_p \\ e_o \end{bmatrix}.$$

前三维 e_p 是位置误差：

$$e_p = p_{\text{target}} - p_{\text{current}}.$$

后三维 e_o 是姿态误差。代码先算

$$R_{\text{err}} = R_{\text{target}} R_{\text{current}}^T,$$

再把误差旋转矩阵转换为“旋转轴乘旋转角”的三维向量。

2.6.2 小运动近似与雅可比

在关节角变化很小时：

$$\Delta x \approx J(q) \Delta q.$$

希望 $\Delta \mathbf{x}$ 抵消当前误差，即

$$\mathbf{J}\Delta \mathbf{q} \approx \mathbf{e}.$$

如果直接使用 $\Delta \mathbf{q} = \mathbf{J}^{-1}\mathbf{e}$ ，会遇到两个问题： $\mathbf{J}$ 可能不可逆，接近奇异位形时数值会爆炸。

2.6.3 阻尼最小二乘

DLS 使用

$$\Delta \mathbf{q} = \mathbf{J}^T (\mathbf{J}\mathbf{J}^T + \lambda^2 \mathbf{I})^{-1} \mathbf{e}.$$

其中 $\lambda > 0$ 是阻尼系数。 $\lambda^2 \mathbf{I}$ 像给病态矩阵加一层缓冲，使其更容易求解。

代码没有写 `inv(A)`，而是：

```
1 A = J * J' + (opts.lambda^2) * eye(6);
2 dq = J' * (A \ e);
```

因为 `A \ e` 会选择更稳定的线性方程求解方法。

更新规则是

$$\mathbf{q}_{k+1} = \mathbf{q}_k + \Delta \mathbf{q}^T.$$

迭代直到 $\|\mathbf{e}\|$ 小于阈值，或达到最大迭代次数。

2.6.4 阻尼系数的影响

- λ 太小：远离奇异点时收敛快，但奇异附近可能出现巨大步长；
- λ 太大：更稳定，但每次只走很小一步，收敛慢且可能残留误差；
- 当前单次 IK 默认常用 0.01，多初值封装中使用 0.05。

2.7 几何雅可比如何推导

2.7.1 雅可比的尺寸

六自由度机械臂的几何雅可比为

$$\mathbf{J} = \begin{bmatrix} \mathbf{J}_v \\ \mathbf{J}_\omega \end{bmatrix} \in \mathbb{R}^{6 \times 6}.$$

它把关节速度映射为末端速度：

$$\begin{bmatrix} \mathbf{v} \\ \boldsymbol{\omega} \end{bmatrix} = \mathbf{J}(\mathbf{q})\dot{\mathbf{q}}.$$

2.7.2 转动关节的一系列

第 i 个转动关节的轴方向为 $\mathbf{z}_i$ ，轴上一点为 $\mathbf{p}_i$ ，末端为 $\mathbf{p}_n$ 。该关节单位角速度引起：

$$\mathbf{J}_{v,i} = \mathbf{z}_i \times (\mathbf{p}_n - \mathbf{p}_i), \quad \mathbf{J}_{\omega,i} = \mathbf{z}_i.$$

2.7.3 MDH 中为什么不能直接拿累计矩阵的 z 轴

本项目单步顺序为

$$\mathbf{R}_x(\alpha)\mathbf{T}_x(a)\mathbf{R}_z(\theta)\mathbf{T}_z(d).$$

真正的关节转轴是执行前两个固定变换之后、执行 $\mathbf{R}_z(\theta)$ 之前的 z 轴。所以代码先计算

$$\mathbf{T}_{\text{pre}} = \mathbf{T} \mathbf{R}_x(\alpha_i) \mathbf{T}_x(a_i),$$

再从 $\mathbf{T}_{\text{pre}}(1:3,3)$ 取轴方向，从 $\mathbf{T}_{\text{pre}}(1:3,4)$ 取轴上一点。

2.8 姿态误差的轴角表示

任意旋转矩阵对应一个旋转轴 $\mathbf{u}$ 和角度 θ 。角度由

$$\cos \theta = \frac{\text{tr}(\mathbf{R}) - 1}{2}$$

得到。旋转轴乘角度为

$$\boldsymbol{\omega} = \frac{\theta}{2 \sin \theta} \begin{bmatrix} R_{32} - R_{23} \\ R_{13} - R_{31} \\ R_{21} - R_{12} \end{bmatrix}.$$

当 θ 很小时，分母也很小，因此代码直接返回零向量，避免数值除零。

2.9 多初值为什么能找到多组解

DLS 是局部方法：从哪个初始点出发，通常会落入附近的某一个解。`build_seed_set` 先准备若干人为设计的肩、肘、腕构型，再在关节限位中生成 30 组固定随机种子。每个种子独立运行 DLS，最终可能到达不同解。

候选解需经过四道筛选：

1. 数值迭代必须声明收敛；
2. 正运动学回代的位置和姿态误差必须足够小；
3. 必须满足关节限位；
4. 与已有解的周期角距离大于阈值，否则视为重复解。

2.10 角度归一化与解的选择

角度具有周期性：

$$q \equiv q + 2k\pi.$$

代码用

$$\text{wrap}(x) = \text{mod}(x + \pi, 2\pi) - \pi$$

把角度差折算到 $[-\pi, \pi)$ 。

选解函数先过滤关节限位，再选择离参考构型最近者：

$$k^* = \arg \min_k \|\text{wrap}(\mathbf{q}^{(k)} - \mathbf{q}_{\text{ref}})\|_2.$$

这样做的工程意义是减少关节突然大幅跳动，保持动作连续。

2.11 当前实现的优点与局限

2.11.1 优点

- 不依赖严格球腕结构，可直接适配当前完整 MDH 模型；
- 每个解都经过 FK 反校验，结果可信；
- DLS 在奇异附近比普通伪逆稳定；
- 多初值可以得到多组候选解。

2.11.2 局限

- 不能数学保证找到全部解，也不能严格保证最多恰好八组；
- 计算量明显高于闭式解析解；
- 收敛依赖初值、阻尼和迭代上限；
- 位置以 mm、姿态以 rad 混在同一误差向量中，未设置尺度权重；
- 接近旋转角 π 时，简单轴角公式也可能数值敏感。

报告答辩建议

不要说“代码已经完整实现 Pieper 闭式解析法”。更准确的表述是：“报告学习并推导了 Pieper 解耦思想；由于当前 SR3 MDH 末端偏置与简化球腕模板不完全一致，工程代码采用多初值 DLS，并用 FK 严格反校验候选解。”

第三章 项目三：路径规划、轨迹生成与跟踪

3.1 路径、轨迹、控制不是同一个东西

路径 只回答“经过哪些构型”，不包含时间。

轨迹 回答“每个时刻处于哪个构型、速度和加速度是多少”。

控制 根据期望轨迹和实际状态的误差，产生驱动命令。

项目三的完整流程是

$$\text{scene} \xrightarrow{\text{plan_path}} \text{path} \xrightarrow{\text{generate_trajectory}} \text{traj} \xrightarrow{\text{simulate_tracking}} \text{result}.$$

3.2 为什么选择关节空间规划

六轴机械臂的一个状态就是六维向量 $\mathbf{q}$ 。关节空间规划直接从 $\mathbf{q}_{\text{start}}$ 搜索到 $\mathbf{q}_{\text{goal}}$ 。

优点是：

- 每个搜索节点天然是一组完整关节角；
- 容易检查关节限位；
- 不需要对每个任务空间采样点反复求 IK；
- 多解跳变问题较少。

缺点是六维空间难以直观绘制，而且关节空间中的直线映射到任务空间后通常是弯曲的，仍必须通过正运动学做碰撞检测。

3.3 场景数据结构

`scene` 中最重要的字段为：

$\mathbf{q}_{\text{start}}, \quad \mathbf{q}_{\text{goal}}, \quad \text{obstacles}, \quad T_{\text{total}}, \quad dt.$

每个球障碍物写成一行

$[c_x, c_y, c_z, r].$

3.4 碰撞检测：为什么不能只看末端

机械臂末端绕过球，不代表手臂中间的连杆也绕过球。因此代码先通过函数 `forward_kinematics` 获得所有坐标系原点：

$$\mathbf{p}_0, \mathbf{p}_1, \dots, \mathbf{p}_6.$$

再把相邻点之间的线段近似为机械臂连杆。

3.4.1 点到线段最短距离

线段端点为 A, B ，球心为 C 。令

$$\mathbf{AB} = \mathbf{B} - \mathbf{A}.$$

把 $C - A$ 投影到 $\mathbf{AB}$ 上：

$$t_0 = \frac{(\mathbf{C} - \mathbf{A})^T \mathbf{AB}}{\mathbf{AB}^T \mathbf{AB}}.$$

如果 $t_0 < 0$ ，最近点是 A ；如果 $t_0 > 1$ ，最近点是 B ；否则最近点在线段内部。因此

$$t = \max(0, \min(1, t_0)), \quad \mathbf{P} = \mathbf{A} + t\mathbf{AB}.$$

最短距离为

$$d = \|\mathbf{P} - \mathbf{C}\|.$$

若 $d < r$ ，线段进入球体，判为碰撞。

3.4.2 状态合法不等于边合法

两个端点都不碰撞，中间仍可能穿过障碍。因此对边 $\mathbf{q}_a \rightarrow \mathbf{q}_b$ 做插值：

$$\mathbf{q}(s) = (1 - s)\mathbf{q}_a + s\mathbf{q}_b, \quad s \in [0, 1].$$

当前规划器取 25 个 s 值逐个检查。采样越密越安全，但计算越慢。

离散检测不是连续数学保证

25 个采样点全部安全，只能说明这些采样点安全。若机械臂在两个采样点之间快速穿过很小的障碍，仍可能漏检。工程上可减小关节插值步长、按连杆最大位移自适应采样，或使用连续碰撞检测。

3.5 先试直线为什么合理

如果起点到终点的关节空间直线本身无碰撞，就没有必要运行随机搜索。直线路径只需两个路点，路径短、计算快、结果稳定。

因此 `plan_path` 首先调用 `is_edge_valid(q_start, q_goal, ...)`。只有直线失败才启动 BiRRT。

3.6 RRT 的基本思想

RRT 是快速扩展随机树。可以把它想成从起点长出一棵树：

1. 在关节限位盒子中随机选一个点 $\mathbf{q}_{\text{rand}}$;
2. 在树中寻找离它最近的节点 $\mathbf{q}_{\text{near}}$;
3. 从最近节点朝随机点走一个固定步长, 得到 $\mathbf{q}_{\text{new}}$;
4. 如果新状态和连接边都安全, 就把新节点加入树;
5. 重复直到树靠近目标。

3.6.1 最近邻

代码使用普通欧氏距离：

$$d_k = \|\mathbf{q}_k - \mathbf{q}_{\text{rand}}\|_2^2.$$

平方不影响最近节点是谁, 同时省去开平方。

3.6.2 Steer

设

$$\boldsymbol{\delta} = \mathbf{q}_{\text{to}} - \mathbf{q}_{\text{from}}.$$

若距离小于步长, 直接到目标; 否则只前进一步:

$$\mathbf{q}_{\text{new}} = \mathbf{q}_{\text{from}} + \Delta q \frac{\boldsymbol{\delta}}{\|\boldsymbol{\delta}\|}.$$

3.7 为什么使用双向 RRT-Connect

普通 RRT 只从起点生长。双向方法同时建立:

$$\mathcal{T}_a \text{ 从起点生长, } \mathcal{T}_b \text{ 从终点生长.}$$

一棵树扩出新节点后, 另一棵树不断朝它延伸, 直到碰撞或成功连接。每轮再交换两棵树的角色。两边同时长, 通常比单向搜索更快相遇。

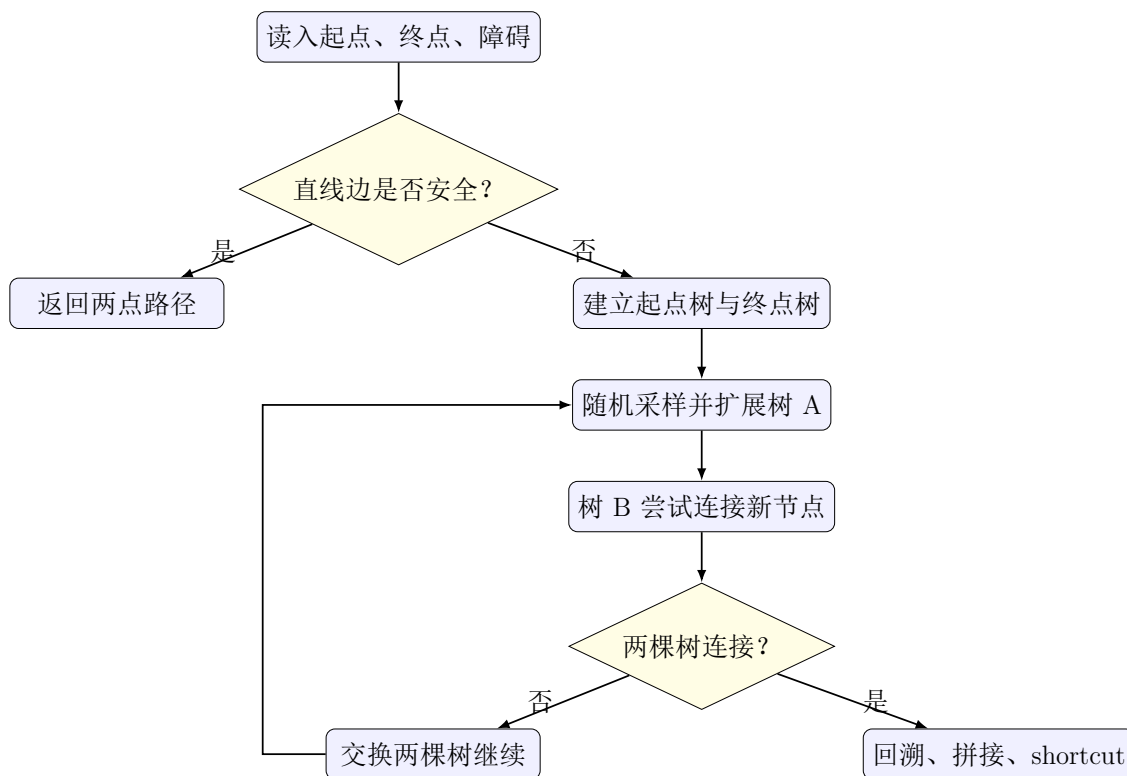


图 3.1: 当前路径规划器的总体流程

3.8 树的数据结构和路径回溯

每棵树保存:

- **tree.q:** 每一行是一个节点;
- **tree.parent:** 每个节点的父节点编号。

新节点加入时记录它从哪个节点扩展而来。连接成功后, 从连接节点沿 **parent** 一直回到根, 就得到半条路径。两棵树各回溯一次, 再翻转并拼接。

3.9 目标偏置与确定性

完全随机采样可能长时间不朝目标前进。当前代码每 5 次迭代直接令

$$\mathbf{q}_{\text{rand}} = \mathbf{q}_{\text{goal}},$$

这叫目标偏置。

代码固定 `rng(3, 'twister')`, 所以同一场景通常得到相同结果, 便于测试、调试和报告复现。但固定种子不等于算法对所有新场景都可靠。

3.10 当前关键参数为什么这样写

参数	当前值	单位	作用
最大迭代次数	6000	次	防止搜索无限运行
步长	0.12 ~ 0.35	rad	太大易撞，太小搜索慢
连接阈值	等于步长	rad	判断两树是否足够接近
边采样数	25	个	检查插值边的碰撞
目标偏置	每 5 次	—	提高向目标生长概率
随机种子	3	—	保证结果可复现

3.11 Shortcut 路径简化

RRT 原始路经常有很多锯齿路点。若路径中的第 i 点可以安全直连更后面的第 j 点，就删除二者之间的多余点：

$$[q_1, \dots, q_i, q_{i+1}, \dots, q_j, \dots] \rightarrow [q_1, \dots, q_i, q_j, \dots].$$

当前实现从最远的 j 开始尝试，因此倾向于一次删掉尽可能多的节点。

3.12 手工绕行兜底

若 6000 次随机搜索仍失败，代码围绕中间构型对第 2、3、5 等关节施加以下几种正负偏移：

$$\frac{\pi}{6}, \quad \frac{\pi}{4}, \quad \frac{\pi}{3}, \quad \frac{\pi}{2}.$$

随后尝试若干三点或四点路径。这能提高给定课程场景下的成功率，但带有明显启发式，不是通用完备方法。

3.13 路径弧长参数化

得到路点 $q_1, \dots, q_M$ 后，先算每段长度：

$$\ell_i = \|q_{i+1} - q_i\|_2.$$

总长度

$$L = \sum_{i=1}^{M-1} \ell_i.$$

路点对应的归一化累计弧长为

$$u_1 = 0, \quad u_i = \frac{\sum_{j=1}^{i-1} \ell_j}{L}, \quad u_M = 1.$$

这样长路径段会分配更多参数区间，短路径段分配更少。

3.14 五次多项式时间缩放从哪里来

希望标量进度 $s(t)$ 满足六个边界条件：

$$s(0) = 0, \quad s(T) = 1,$$

$$\dot{s}(0) = \dot{s}(T) = 0,$$

$$\ddot{s}(0) = \ddot{s}(T) = 0.$$

六个条件至少需要六个系数，所以取五次多项式。令

$$\tau = \frac{t}{T}, \quad s(\tau) = a_0 + a_1\tau + a_2\tau^2 + a_3\tau^3 + a_4\tau^4 + a_5\tau^5.$$

代入六个边界条件并解线性方程，得到

$$a_0 = a_1 = a_2 = 0, \quad a_3 = 10, \quad a_4 = -15, \quad a_5 = 6,$$

因此

$$s(\tau) = 10\tau^3 - 15\tau^4 + 6\tau^5.$$

导数为

$$\begin{aligned} \dot{s}(t) &= \frac{30\tau^2 - 60\tau^3 + 30\tau^4}{T}, \\ \ddot{s}(t) &= \frac{60\tau - 180\tau^2 + 120\tau^3}{T^2}. \end{aligned}$$

3.15 从标量进度得到每个关节的轨迹

在某个时刻令 $u = s(t)$ ，先确定 u 落在哪个路径段 $[u_i, u_{i+1}]$ 。局部比例为

$$\lambda = \frac{u - u_i}{u_{i+1} - u_i}.$$

位置线性插值：

$$\mathbf{q}(t) = \mathbf{q}_i + \lambda(\mathbf{q}_{i+1} - \mathbf{q}_i).$$

该段对 u 的导数为

$$\frac{d\mathbf{q}}{du} = \frac{\mathbf{q}_{i+1} - \mathbf{q}_i}{u_{i+1} - u_i}.$$

链式法则给出

$$\dot{\mathbf{q}}(t) = \frac{d\mathbf{q}}{du} \dot{s}(t), \quad \ddot{\mathbf{q}}(t) = \frac{d\mathbf{q}}{du} \ddot{s}(t).$$

当前轨迹并非在路点处完全光滑

全局 $s(t)$ 很光滑，但 $\mathbf{q}(u)$ 在路点处是分段线性的，方向可能突然改变。因此 $d\mathbf{q}/du$ 在中间路点处可能跳变，关节速度和加速度也可能不连续。当前实现满足课程测试的首末边界与数值差分要求，但严格意义上不是全程 C^2 平滑轨迹。改进可使用样条、分段五次多项式或 min-jerk via-point。

3.16 PD 控制器

动态模式控制律为

$$\boldsymbol{\tau} = \mathbf{K}_p \odot (\mathbf{q}_d - \mathbf{q}) + \mathbf{K}_d \odot (\dot{\mathbf{q}}_d - \dot{\mathbf{q}}) + \ddot{\mathbf{q}}_d.$$

其中 $\odot$ 表示逐关节相乘。

- 比例项像弹簧，位置偏差越大，纠正越强；
- 微分项像阻尼，抑制速度误差和振荡；
- 前馈加速度提前告诉系统期望怎么加速。

当前仿真假设等效惯性矩阵为单位阵，忽略重力、科氏力、摩擦：

$$\ddot{\mathbf{q}} = \boldsymbol{\tau}.$$

这只是教学用简化模型， $\boldsymbol{\tau}$ 在这里更像“加速度命令”，不能直接等同真实电机力矩。

3.17 Euler 积分

每个时间步长 Δt ：

$$\dot{\mathbf{q}}_{k+1} = \dot{\mathbf{q}}_k + \ddot{\mathbf{q}}_k \Delta t,$$

$$\mathbf{q}_{k+1} = \mathbf{q}_k + \dot{\mathbf{q}}_{k+1} \Delta t.$$

代码先更新速度再更新位置，属于半隐式形式。步长太大会导致数值不稳定。

3.18 项目三的局限与改进方向

1. 连杆被近似为无厚度线段，真实机械臂有外壳半径。可把障碍半径扩大安全距离，或使用胶囊体碰撞模型。
2. 随机规划不保证在有限次数内成功。可使用 RRT-Connect 的成熟实现、PRM 或 RRT*。
3. 六个关节直接用同一欧氏度量，未考虑不同关节运动对末端影响不同。可设置关节权重。
4. shortcut 只减少路点，不直接优化安全余量和曲率。

5. 轨迹未自动检查速度、加速度峰值。可根据限值自动放大 T 。
6. 动态模型过于简化。真实控制应考虑 $M(\mathbf{q})\ddot{\mathbf{q}} + C(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + g(\mathbf{q}) + f(\dot{\mathbf{q}})$ 。

第四章 代码阅读、测试判据与答辩准备

4.1 项目二逐文件阅读顺序

1. `robot_params.m`: 先确认机器人有几轴、尺寸、偏置和限位。
2. `build_mdh_table.m`: 理解 q 如何变成 θ 。
3. `mdh_transform.m`: 确认采用 MDH, 而不是标准 DH。
4. `forward_kinematics.m`: 理解六个局部变换如何累计。
5. `jacobian_geometric.m`: 理解小关节运动如何影响末端。
6. `inverse_kinematics_numerical.m`: 理解 DLS 单次迭代。
7. `inverse_kinematics_analytical.m`: 理解多初值搜索、校验和去重。
8. `select_ik_solution.m`: 理解多解中如何挑一组连续、安全的解。

4.2 项目二核心代码逐句解释

4.2.1 正运动学累计

```
1 T = eye(4);  
2 for i = 1:params.n  
3     A_i = mdh_transform(a_i, alpha_i, d_i, theta_i);  
4     T = T * A_i;  
5     T_all(:, :, i + 1) = T;  
6 end
```

- `eye(4)` 表示从基坐标系到自身不发生变化;
- `A_i` 是第 i 节局部变换;
- 右乘保持坐标变换链的正确顺序;
- 保存 `T_all` 是为了后续使用所有关节位置。

4.2.2 MDH 关节轴

```
1 T_pre = T * rotx4(alpha_i) * transl4(a_i, 0, 0);
```

```

2 z_axis = T_pre(1:3, 3);
3 p_axis = T_pre(1:3, 4);
4 Jv = cross(z_axis, p_end - p_axis);
5 Jw = z_axis;

```

T_{pre} 的含义是“已经做完固定的扭转和平移，但还没有执行当前关节的变量旋转”。因此它的 z 轴就是当前转动关节轴。

4.2.3 DLS 更新

```

1 ep = T_target(1:3, 4) - T_cur(1:3, 4);
2 R_err = T_target(1:3, 1:3) * T_cur(1:3, 1:3)';
3 eo = rotm_to_axisangle(R_err);
4 e = [ep; eo];
5
6 J = jacobian_geometric(q, params);
7 A = J * J' + (opts.lambda^2) * eye(6);
8 dq = J' * (A \ e);
9 q = q + dq(:)';

```

这段代码每轮完成“测误差、算灵敏度、求修正量、更新猜测”四件事。

4.3 项目三逐文件阅读顺序

1. `scene_easy.m`、`scene_hard.m`：明确输入是什么。
2. `plan_path.m` 主函数：先看整体分支。
3. `is_collision` 和 `seg_point_dist`：理解安全判据。
4. `extend_tree`、`connect_tree`：理解 BiRRT。
5. `merge_trees`、`shortcut_path`：理解路径恢复和简化。
6. `generate_trajectory.m`：理解时间缩放和链式法则。
7. `pd_controller.m`、`simulate_tracking.m`：理解理想回放与简化动态跟踪的区别。

4.4 项目三核心代码逐句解释

4.4.1 直线优先

```

1 if is_edge_valid(q_start, q_goal, scene, params, qlim, edge_samples)
2     path = [q_start; q_goal];
3 else
4     path = plan_with_birrt(...);

```



```
5 end
```

这里不是“偷懒”，而是合理的分层策略：简单问题直接解决，困难问题才使用搜索。

4.4.2 扩展一步

```
1 idx_near = nearest_node(tree.q, q_target);
2 q_near = tree.q(idx_near, :);
3 q_new = steer(q_near, q_target, step);
4 added = is_state_valid(q_new, ...) && ...
5         is_edge_valid(q_near, q_new, ...);
```

新节点必须同时满足“点安全”和“从父节点走到它的整条边安全”。

4.4.3 轨迹时间缩放

```
1 tau = traj.t / scene.T_total;
2 s = 10*tau.^3 - 15*tau.^4 + 6*tau.^5;
3 ds_dt = (30*tau.^2 - 60*tau.^3 + 30*tau.^4) ...
4         / scene.T_total;
5 d2s_dt2 = (60*tau - 180*tau.^2 + 120*tau.^3) ...
6          / (scene.T_total^2);
```

`.*` 和 `.^` 表示对时间数组逐元素运算。若误写成矩阵乘方；MATLAB 会报尺寸错误或得到完全不同的结果。

4.5 测试到底在检查什么

4.5.1 IK 自洽测试

测试不是简单比较 $\mathbf{q}' = \mathbf{q}$ ，因为逆运动学可以有多解。真正判据是：

$$\text{FK}(\mathbf{q}') \approx \text{FK}(\mathbf{q}),$$

并且解集中至少有一组与原始 $\mathbf{q}$ 周期等价。

4.5.2 雅可比中心差分测试

对第 i 个关节分别加减很小扰动 h ：

$$\frac{\partial f}{\partial q_i} \approx \frac{f(\mathbf{q} + h\mathbf{e}_i) - f(\mathbf{q} - h\mathbf{e}_i)}{2h}.$$

中心差分误差通常比单边差分小。测试将解析雅可比与数值雅可比的 Frobenius 范数误差比较。

4.5.3 路径测试

依次检查：

1. 矩阵尺寸是否为 $M \times 6$;
2. 第一个点是否等于起点;
3. 最后一个点是否等于终点;
4. 所有路点是否在关节限位内;
5. 每条边插值后, 所有连杆是否避开所有球。

4.5.4 轨迹测试

检查字段尺寸、等间隔时间、端点位置、端点速度、端点加速度, 以及

$$\frac{\mathbf{q}_{k+1} - \mathbf{q}_k}{dt}$$

是否与相邻解析速度的平均值基本一致。

4.5.5 跟踪测试

运动学模式直接令实际关节角等于期望关节角, 因此理论误差为零。动态模式才会产生跟踪误差, 需要通过 K_p, K_d 调节。

4.6 推荐的实验结果图

课程题目要求的图建议至少包括:

1. easy 与 hard 场景三维图: 障碍球、起点、终点、中间路点和末端轨迹;
2. 六个关节角 $q_i(t)$ 曲线;
3. 六个关节速度 $\dot{q}_i(t)$ 曲线;
4. 六个关节加速度 $\ddot{q}_i(t)$ 曲线;
5. 机械臂到最近障碍球面的最小安全距离曲线;
6. 动态模式下关节误差或末端位置误差曲线。

不要凭文字编造数值。应运行当前代码后把真实图和真实测试输出加入最终提交版。

4.7 最小安全距离如何计算

对每个时刻、每个障碍、每根连杆计算线段到球心距离 $d_{j,k}$ 。球面净空为

$$c_{j,k} = d_{j,k} - r_k.$$

该时刻全机械臂最小安全距离：

$$c_{\min}(t) = \min_{j,k} c_{j,k}(t).$$

若 $c_{\min} > 0$ ，说明有正安全余量；若等于零，刚好接触；若小于零，发生穿透。

4.8 常见报错与定位

现象	优先检查
FK 结果完全不对	MDH/标准 DH 是否混用，offset 是否漏加，矩阵顺序是否写反
雅可比位置部分错误	是否使用了 MDH 正确关节轴，叉乘顺序是否反了
DLS 不收敛	初值、阻尼、姿态误差、单位权重、雅可比是否正确
IK 有正确位姿但角度不同	多解和 2π 周期，不应直接逐元素硬比较
RRT 一直失败	起终点是否已碰撞、步长、边采样、障碍是否封死通道
路径端点安全但中间撞球	只检查了状态，没有检查边
轨迹速度与差分不一致	链式法则、段索引、 u 节点归一化是否正确
动态跟踪振荡	K_p 过大、 K_d 过小、Euler 步长过大

4.9 答辩可能提问与简洁回答

为什么用关节空间 RRT，而不是任务空间直线？

关节空间节点天然满足完整六轴构型表达，容易处理关节限位；任务空间路径需要频繁求 IK，并可能在不同 IK 分支间跳变。碰撞仍通过 FK 映射到任务空间检查。

为什么用双向 RRT？

起点树和终点树同时扩展，通常更快连接，尤其适合已知单一起点和终点的问题。

为什么要 shortcut?

RRT 路径受随机采样影响，节点多且绕。shortcut 用无碰撞长边替换若干短边，减少路点和路径长度。

为什么五次多项式能保证首末速度、加速度为零?

因为它的六个系数正好由位置、速度、加速度的六个首末边界条件唯一确定。

DLS 比普通伪逆好在哪里?

加入 $\lambda^2 I$ 后，雅可比接近奇异时求解仍较稳定，不容易产生过大的关节修正量。

你们的“解析 IK”是真正解析法吗?

当前运行代码不是严格闭式解析解，而是多初值 DLS 数值法。报告保留 Pieper 分离法作为理论学习，并明确说明当前完整 SR3 MDH 模型使用数值反解和 FK 校验。

4.10 AI 工具使用声明模板

本项目在学习、文档组织和代码解释过程中使用了 OpenAI Codex 辅助工具。主要用途包括：梳理 MDH、雅可比、DLS、BiRRT 和五次时间缩放的基础原理；把仓库中的 MATLAB 函数按调用关系整理为可阅读的说明；检查 LaTeX 公式、结构和编译问题。

具体示例包括：

1. 辅助解释为什么 MDH 下雅可比关节轴应在 $\mathbf{R}_x(\alpha)\mathbf{T}_x(a)$ 之后提取；
2. 辅助解释 DLS 中阻尼项的数值作用，以及为什么使用 MATLAB 反斜杠而不是显式求逆；
3. 辅助整理 BiRRT、碰撞检测、shortcut 和五次时间缩放的逐步说明。

最终提交前，小组成员应自行核对公式与当前代码的一致性，亲自运行测试、生成实验图、记录参数与结果，并能够在现场不依赖 AI 独立解释算法和代码。算法选择、实际调参、测试结果和答辩内容应由小组成员共同确认。

附录 A 核心 MATLAB 源码附录

本附录通过 `\lstinputlisting` 直接读取仓库中的当前源码。因此代码更新后重新编译，附录也会同步更新。

A.1 项目二核心代码

A.1.1 机器人参数

```
1 function params = robot_params(robot_type)
2 %ROBOT_PARAMS 返回所选机械臂的结构参数
3 %
4 %   params = robot_params(robot_type)
5 %
6 %   输入:
7 %       robot_type : 字符串或字符数组
8 %                   可选 'CR7' 或 'SR3'
9 %
10 %   输出:
11 %       params      : 机械臂参数结构体
12 %
13 %   说明:
14 %       本函数是整个项目统一的参数入口。
15 %       后续的轨迹生成、仿真测试等项目建议继续调用本函数，
16 %       因此不建议随意修改函数名称和输入输出接口。
17
18 if isstring(robot_type)
19     robot_type = char(robot_type);
20 end
21 % 根据小组选择的机器人型号填写
22 switch upper(robot_type)
23     case 'CR7'
24         params.name = 'CR7';
25         params.n = 6;
26
27         % TODO:
```

```

28 % 根据 CR7 的结构图和参数表，填写以下 MDH/几何参数。
29 % ! ! ! 下方数值仅为占位示例，不代表真实机械臂参数。
30 params.a      = [0, 300, 250, 0, 0, 0];
31 params.alpha  = [pi/2, 0, 0, pi/2, -pi/2, 0];
32 params.d      = [300, 0, 0, 200, 0, 80];
33 params.offset = [0, 0, 0, 0, 0, 0];
34
35 % 可选：关节范围，后续做轨迹规划和仿真时可以继续使用
36 params.qlim = repmat([-pi, pi], 6, 1);
37
38 case 'SR3'
39     params.name = 'SR3';
40     params.n = 6;
41
42     % SR3 MDH 参数。
43     % 每一行参数为 [a_{i-1}, alpha_{i-1}, d_i, theta_i]。
44     params.a      = [0, 0, 290, 0, 0, 136];
45     params.alpha  = [0, pi/2, pi, pi/2, -pi/2, pi/2];
46     params.d      = [344, 0, 0, 290, 0, 103.5];
47     params.offset = [pi, pi/2, pi/2, pi, 0, 0];
48
49     params.qlim = [
50         -pi, pi;
51         -130*pi/180, 130*pi/180;
52         -150*pi/180, 150*pi/180;
53         -pi, pi;
54         -120*pi/180, 120*pi/180;
55         -2*pi, 2*pi
56     ];
57     otherwise
58         error('不支持的机械臂型号，请选择 ''CR7'' 或 ''SR3''。');
59 end
60 end

```

Listing A.1: robot_params.m

A.1.2 MDH 参数表构造

```

1 function mdh_table = build_mdh_table(q, params)
2 %BUILD_MDH_TABLE 根据关节变量和机械臂参数生成 MDH 参数表
3 %
4 %   mdh_table = build_mdh_table(q, params)
5 %
6 %   输入：

```

```

7 %      q      : 1xn 关节变量向量, 单位为弧度
8 %      params  : 由 robot_params() 返回的机械臂参数结构体
9 %
10 % 输出:
11 %      mdh_table : n×4 的参数表, 每一行为
12 %                [a_{i-1}, alpha_{i-1}, d_i, theta_i]
13 %
14 % 说明:
15 % 1. 本模板默认所有关节均为转动关节。
16 % 2. 如果机械臂中包含移动关节, 可在此基础上自行扩展。
17 % 3. 本项目采用改进 DH (Modified DH / MDH), 与 RTB 中
18 %    Link(..., 'modified') 的参数顺序保持一致。
19 % 4. 建议后续项目继续保持本函数接口不变。
20
21 q = q(:)'; % 保证 q 为行向量
22
23 if length(q) ~= params.n
24     error('输入的关节变量维度与机械臂自由度不一致。');
25 end
26
27 mdh_table = zeros(params.n, 4);
28
29 for i = 1:params.n
30     a_i      = params.a(i);
31     alpha_i  = params.alpha(i);
32     d_i      = params.d(i);
33
34     % MDH 转动关节: theta_i = q_i + offset_i。
35     % offset_i 表示电机零位到建模零位的角度修正, 不是固定姿态角。
36     theta_i = q(i) + params.offset(i);
37
38     mdh_table(i, :) = [a_i, alpha_i, d_i, theta_i];
39 end
40 end

```

Listing A.2: build_mdh_table.m

A.1.3 单节 MDH 变换

```

1 function T = mdh_transform(a, alpha, d, theta)
2 %MDH_TRANSFORM 根据单节 MDH 参数生成齐次变换矩阵
3 %
4 % T = mdh_transform(a, alpha, d, theta)
5 %

```

```

6 % 输入：
7 %     a, alpha, d, theta : 一组 MDH 参数
8 %
9 % 输出：
10 %     T : 4×4 齐次变换矩阵
11 %
12 % 说明：
13 % 本函数采用改进 DH (Modified DH / MDH) :
14 %     A_i = Rx(alpha_{i-1}) * Tx(a_{i-1}) * Rz(theta_i) * Tz(d_i)
15 % 必须保证报告中的公式推导与这里的代码表达一致。
16
17 ct = cos(theta);
18 st = sin(theta);
19 ca = cos(alpha);
20 sa = sin(alpha);
21
22 T = [ ct,    -st,    0,    a;
23       st*ca, ct*ca, -sa,  -d*sa;
24       st*sa, ct*sa,  ca,   d*ca;
25       0,     0,     0,    1 ];
26
27 % 如果学生采用的是标准 DH, 请在此处修改,
28 % 并在课程设计报告中明确说明。
29 end

```

Listing A.3: mdh_transform.m

A.1.4 正运动学

```

1 function [T_end, T_all] = forward_kinematics(q, params)
2 %FORWARD_KINEMATICS 计算机械臂正运动学
3 %
4 % [T_end, T_all] = forward_kinematics(q, params)
5 %
6 % 输入：
7 %     q      : 1xn 关节变量向量
8 %     params : 机械臂参数结构体
9 %
10 % 输出：
11 %     T_end   : 末端执行器相对于基坐标系的 4×4 齐次变换矩阵
12 %     T_all   : 4×4×(n+1) 的三维数组, 保存各级累计变换矩阵
13 %
14 % 说明：
15 %     T_all(:, :, 1) 为基坐标系变换矩阵, 等于 eye(4)

```



```

16 % T_all(:, :, i+1) 表示基坐标系到第 i 个关节坐标系的累计变换
17
18 mdh_table = build_mdh_table(q, params);
19
20 T = eye(4);
21 T_all = zeros(4, 4, params.n + 1);
22 T_all(:, :, 1) = T;
23
24 for i = 1:params.n
25     a_i      = mdh_table(i, 1);
26     alpha_i  = mdh_table(i, 2);
27     d_i      = mdh_table(i, 3);
28     theta_i  = mdh_table(i, 4);
29
30     % 计算当前连杆的单步变换矩阵
31     A_i = mdh_transform(a_i, alpha_i, d_i, theta_i);
32
33     % 更新累计变换矩阵
34     T = T * A_i;
35     T_all(:, :, i + 1) = T;
36 end
37
38 T_end = T;
39 end

```

Listing A.4: forward_kinematics.m

A.1.5 几何雅可比

```

1 function J = jacobian_geometric(q, params)
2 %JACOBIAN_GEOMETRIC 计算机械臂在关节角 q 处的 6×n 几何雅可比矩阵
3 %
4 % J = jacobian_geometric(q, params)
5 %
6 % 输入:
7 %     q      : 1×n 关节变量向量 (单位: 弧度)
8 %     params : 由 robot_params() 返回的机械臂参数结构体
9 %
10 % 输出:
11 %     J      : 6×n 几何雅可比矩阵
12 %             前 3 行为线速度部分 Jv (米/弧度, 或与 a, d 相同长度单位)
13 %             后 3 行为角速度部分 Jw (无量纲, 即 1/弧度)
14 %
15 % 公式 (转动关节):

```

```

16 %      第 i 列:
17 %      Jv_i = z_{i-1} × (p_n - p_{i-1})
18 %      Jw_i = z_{i-1}
19 %
20 %      其中:
21 %      z_{i-1} : 第 i-1 个连杆坐标系 z 轴在基系下的方向
22 %                (即 T_all(:, :, i) 第 3 列)
23 %      p_{i-1} : 第 i-1 个连杆坐标系原点在基系下的位置
24 %                (即 T_all(:, :, i) 第 4 列)
25 %      p_n      : 末端坐标系原点在基系下的位置
26 %                (即 T_all(:, :, n+1) 第 4 列)
27 %
28 %      说明:
29 %      T_all(:, :, 1) = eye(4)          (基坐标系)
30 %      T_all(:, :, i+1) = base→关节 i 末端的累计变换
31 %
32 %      学生需要完成 2 处【填空】: 写出 Jv、Jw 的表达式。
33
34 % 通过正运动学得到末端位置。
35 % 注意: 本项目采用 MDH, 关节 i 的转轴是执行
36 % Rx(alpha_{i-1}) * Tx(a_{i-1}) 之后、Rz(theta_i) 之前的 z 轴,
37 % 不能直接使用标准 DH 中的 z_{i-1}。
38 T_end = forward_kinematics(q, params);
39
40 n = params.n;
41 p_end = T_end(1:3, 4); % 末端位置
42
43 J = zeros(6, n);
44 mdh_table = build_mdh_table(q, params);
45 T = eye(4);
46
47 for i = 1:n
48     a_i      = mdh_table(i, 1);
49     alpha_i  = mdh_table(i, 2);
50     d_i      = mdh_table(i, 3);
51     theta_i  = mdh_table(i, 4);
52
53     T_pre = T * rotx4(alpha_i) * transl4(a_i, 0, 0);
54     z_axis = T_pre(1:3, 3);
55     p_axis = T_pre(1:3, 4);
56
57     % -----
58     % 【填空 1】 第 i 列的线速度部分 Jv (3×1)
59     % 公式: Jv = z_{i-1} × (p_n - p_{i-1})
60     % 提示: Matlab 内置 cross(a, b) 用于三维叉乘。

```

```

61     % -----
62     Jv = cross(z_axis, p_end - p_axis);
63
64     % -----
65     % 【填空 2】 第 i 列的角速度部分 Jw (3×1)
66     % 公式: Jw = z_{i-1}
67     % -----
68     Jw = z_axis;
69
70     if any(~isfinite(Jv)) || any(~isfinite(Jw))
71         error('几何雅可比失败: TODO 1 / TODO 2 尚未完成。');
72     end
73
74     J(:, i) = [Jv; Jw];
75
76     T = T_pre * rotz4(theta_i) * transl4(0, 0, d_i);
77 end
78 end
79
80
81 function T = rotx4(alpha)
82     ca = cos(alpha);
83     sa = sin(alpha);
84     T = [1, 0, 0, 0;
85          0, ca, -sa, 0;
86          0, sa,  ca, 0;
87          0, 0,  0, 1];
88 end
89
90
91 function T = rotz4(theta)
92     ct = cos(theta);
93     st = sin(theta);
94     T = [ct, -st, 0, 0;
95          st,  ct, 0, 0;
96          0,   0,  1, 0;
97          0,   0,  0, 1];
98 end
99
100
101 function T = transl4(x, y, z)
102     T = [1, 0, 0, x;
103          0, 1, 0, y;
104          0, 0, 1, z;
105          0, 0, 0, 1];

```

106 end

Listing A.5: jacobian_geometric.m

A.1.6 DLS 数值逆运动学

```

1 function [q, info] = inverse_kinematics_numerical(T_target, q0, params, opts)
2 %INVERSE_KINEMATICS_NUMERICAL 基于阻尼最小二乘法 (DLS) 的数值逆运动学
3 %
4 % [q, info] = inverse_kinematics_numerical(T_target, q0, params, opts)
5 %
6 % 输入:
7 %     T_target : 4x4 末端目标位姿
8 %     q0       : 1xn 初始关节角 (迭代起点)
9 %     params   : 机械臂参数结构体
10 %     opts     : (可选) struct, 可包含字段
11 %                .max_iter 最大迭代次数 (默认 200)
12 %                .tol      收敛判据 (默认 1e-6)
13 %                .lambda   DLS 阻尼系数 (默认 0.01)
14 %                .verbose  是否打印每步迭代误差 (默认 false)
15 %
16 % 输出:
17 %     q        : 1xn 收敛后的关节角
18 %     info     : struct, 包含
19 %                .iter      实际迭代次数
20 %                .err_hist  每步 ||e|| 历史 (列向量)
21 %                .converged 是否在 max_iter 内收敛
22 %
23 % 方法说明 (备选内容):
24 %     本课程主线为解析法逆运动学; 本文件提供基于 Jacobian 的数值
25 %     逆运动学作为对照与扩展, 便于理解"逆向问题为何天然是迭代的",
26 %     同时为 Project 3 的轨迹跟踪/控制做铺垫。
27 %
28 % DLS 迭代公式:
29 %      $\Delta q = J^T (J \cdot J^T + \lambda^2 \cdot I)^{-1} e$ 
30 %     其中  $e = [\text{位置误差}; \text{姿态误差}]$ , 6x1 列向量;
31 %      $\lambda^2 \cdot I$  项保证在奇异附近 ( $J \cdot J^T$ ) 接近退化时仍可求逆。
32 %
33 % 学生需要完成 2 处【填空】。
34
35 if nargin < 4 || isempty(opts); opts = struct(); end
36 if ~isfield(opts, 'max_iter'); opts.max_iter = 200; end
37 if ~isfield(opts, 'tol'); opts.tol = 1e-6; end
38 if ~isfield(opts, 'lambda'); opts.lambda = 0.01; end

```

```

39     if ~isfield(opts, 'verbose');      opts.verbose = false;end
40
41     q = q0(:)';
42     err_hist = zeros(opts.max_iter, 1);
43     converged = false;
44
45     for k = 1:opts.max_iter
46         % 当前末端位姿
47         T_cur = forward_kinematics(q, params);
48
49         % 6×1 位姿误差 e = [位置误差 ; 姿态误差 (轴角形式)]
50         ep = T_target(1:3, 4) - T_cur(1:3, 4);
51         R_err = T_target(1:3, 1:3) * T_cur(1:3, 1:3)';
52         eo = rotm_to_axisangle(R_err);
53         e = [ep; eo];
54
55         err_hist(k) = norm(e);
56         if opts.verbose
57             fprintf(' iter %3d ||e|| = %.6e\n', k, err_hist(k));
58         end
59
60         if err_hist(k) < opts.tol
61             converged = true;
62             break;
63         end
64
65         % 当前几何雅可比 (调用学生写的 jacobian_geometric)
66         J = jacobian_geometric(q, params);
67
68         % -----
69         % 【填空 1】 写出 DLS 增量公式
70         %  $\Delta q = J^T \cdot (J \cdot J^T + \lambda^2 \cdot I)^{-1} \cdot e$ 
71         % 提示:
72         % - I 是 6×6 单位矩阵
73         % - 用 backslash 比 inv() 数值更稳定:
74         %  $A \setminus b$  等价于  $\text{inv}(A) * b$ 
75         % - 期望维度: dq 为 n×1 列向量 (这里 n = 6)
76         % -----
77         I6 = eye(6);
78         A = J * J' + (opts.lambda^2) * I6;
79         dq = J' * (A \ e);
80
81         if any(~isfinite(dq))
82             error('数值 IK 失败: TODO 1 (DLS 增量) 尚未完成。');
83         end

```

```

84
85 % -----
86 % 【填空 2】 用 dq 更新 q
87 % 提示: q 在本函数内是 1×n 行向量, dq 是 n×1 列向量,
88 %       注意维度匹配 (q + dq' 或 q + reshape(dq,1,[]))。
89 % -----
90 q_new = q + dq(:)';
91
92 if any(~isfinite(q_new))
93     error('数值 IK 失败: TODO 2 (q 更新) 尚未完成。');
94 end
95 q = q_new;
96 end
97
98 info.iter      = k;
99 info.err_hist  = err_hist(1:k);
100 info.converged = converged;
101 end
102
103
104 function w = rotm_to_axisangle(R)
105 %ROTM_TO_AXISANGLE 把 3×3 旋转矩阵转成 3×1 轴角向量 (axis × angle)
106 cos_th = (trace(R) - 1) / 2;
107 cos_th = max(-1, min(1, cos_th));
108 th = acos(cos_th);
109 if abs(th) < 1e-9
110     w = [0; 0; 0];
111 else
112     w = th / (2 * sin(th)) * [R(3,2) - R(2,3);
113                               R(1,3) - R(3,1);
114                               R(2,1) - R(1,2)];
115 end
116 end

```

Listing A.6: inverse_kinematics_numerical.m

A.1.7 多初值逆运动学封装

```

1 function Q_all = inverse_kinematics_analytical(T_target, params)
2 %INVERSE_KINEMATICS_ANALYTICAL 求解与当前 SR3 MDH 表一致的逆运动学
3 %
4 %   Q_all = inverse_kinematics_analytical(T_target, params)
5 %
6 %   当前使用的 SR3 MDH 表含 a6 与 d6 工具偏置, 不再是 README 模板中

```

```

7 % 最简单的球腕形式。这里采用多初值 DLS 搜索，并对每个候选解做 FK
8 % 反校验和去重，保持本项目测试所需的多解输出接口。
9
10 if params.n ~= 6
11     error('当前 IK 求解器仅支持 6 自由度机械臂。');
12 end
13
14 seeds = build_seed_set(params);
15 opts = struct('max_iter', 120, ...
16             'tol', 1e-9, ...
17             'lambda', 0.05, ...
18             'verbose', false);
19
20 solutions = [];
21 for i = 1:size(seeds, 1)
22     try
23         [q_sol, info] = inverse_kinematics_numerical(T_target, seeds(i, :),
24             params, opts);
25         catch
26             continue;
27         end
28
29         q_sol = wrap_to_pi(q_sol);
30         if ~info.converged || ~is_valid_solution(q_sol, T_target, params)
31             continue;
32         end
33
34         if isfield(params, 'qlim') && ~isempty(params.qlim)
35             tol = 1e-7;
36             if any(q_sol < params.qlim(:, 1)' - tol) || any(q_sol > params.qlim(:,
37                 2)' + tol)
38                 continue;
39             end
40         end
41
42         if isempty(solutions) || all(vecnorm(wrap_to_pi(solutions - q_sol), 2, 2) >
43             1e-5)
44             solutions(end+1, :) = q_sol; %#ok<AGROW>
45         end
46     end
47
48     Q_all = solutions;
49 end

```

```

49 function seeds = build_seed_set(params)
50 %BUILD_SEED_SET 构造覆盖常见肩/肘/腕构型的确定性初值集合。
51     base_deg = [
52         0    0    0    0    0    0;
53         0 -45  45    0  45    0;
54         0  45 -45    0 -45    0;
55         90    0    0    0  60    0;
56        -90    0    0    0 -60    0;
57         45 -45  45  45  45  45;
58        -45  45 -45 -45 -45 -45;
59        120 -90  90 100  30 120;
60        -90  45 -45 -90  60  45
61    ];
62    seeds = deg2rad(base_deg);
63
64    if isfield(params, 'qlim') && ~isempty(params.qlim)
65        qlim = params.qlim;
66        rng(7);
67        random_seeds = zeros(30, params.n);
68        for i = 1:size(random_seeds, 1)
69            random_seeds(i, :) = qlim(:, 1)' + rand(1, params.n) .* ...
70                (qlim(:, 2)' - qlim(:, 1)');
71        end
72        seeds = [seeds; random_seeds]; %#ok<AGROW>
73    end
74 end
75
76
77 function ok = is_valid_solution(q, T_target, params)
78 %IS_VALID_SOLUTION 用 FK 验证候选解是否真正到达目标位姿。
79     T_chk = forward_kinematics(q, params);
80     pos_err = norm(T_chk(1:3, 4) - T_target(1:3, 4));
81     rot_err = norm(T_chk(1:3, 1:3) - T_target(1:3, 1:3), 'fro');
82     ok = pos_err < 1e-5 && rot_err < 1e-5;
83 end
84
85
86 function w = wrap_to_pi(x)
87 %WRAP_TO_PI 把角度折算到 [-pi, pi]。
88     w = mod(x + pi, 2*pi) - pi;
89 end

```

Listing A.7: inverse_kinematics_analytical.m

A.1.8 逆解选择

```

1 function [q_best, idx] = select_ik_solution(Q_all, q_ref, params)
2 %SELECT_IK_SOLUTION 从多组 IK 解中按某种策略挑选一组
3 %
4 % [q_best, idx] = select_ik_solution(Q_all, q_ref, params)
5 %
6 % 输入:
7 %     Q_all : N×n, 由 inverse_kinematics_analytical 返回的全部解
8 %     q_ref : 1×n 参考关节角 (是否使用取决于你选择的策略)
9 %     params : 机械臂参数结构体 (含 qlim 等)
10 %
11 % 输出:
12 %     q_best : 1×n, 被选中的关节解
13 %     idx : q_best 在 Q_all 中的行号
14 %
15 % =====
16 % 本函数为开放性任务。
17 %
18 % 解析法 IK 通常返回多组解 (CR7 / SR3 这类 6R 球腕臂最多 8 组),
19 % 不同应用场景对"哪一组解最合适"有不同要求。请你根据所选机械臂的
20 % 工作场景, 自行设计选解方案, 并在课程设计报告中说明:
21 %
22 % 1. 选解所依据的指标 (例如关节变化、位形约束、安全余量等等);
23 % 2. 在多解情况下如何取舍;
24 % 3. 设计该方案的理由。
25 %
26 % 函数最低要求:
27 % - 输入合法时返回一行 q_best 与 idx;
28 % - Q_all 为空时报错。
29 % =====
30
31 if isempty(Q_all)
32     error('IK 无解: 传入的 Q_all 为空, 请检查目标位姿是否在工作空间内。');
33 end
34 q_ref = q_ref(:)';
35
36 % 先按关节限位过滤, 再从可行解中选择离参考关节角最近的一组。
37 % 角度差按 2*pi 周期折算, 避免 q 与 q+2*pi 被误判为距离很远。
38 feasible = true(size(Q_all, 1), 1);
39 if isfield(params, 'qlim') && ~isempty(params.qlim)
40     tol = 1e-9;
41     feasible = all(Q_all >= (params.qlim(:, 1)' - tol) & ...
42                   Q_all <= (params.qlim(:, 2)' + tol), 2);
43 end

```

```

44
45     candidate_idx = find(feasible);
46     if isempty(candidate_idx)
47         candidate_idx = (1:size(Q_all, 1))';
48     end
49
50     diffs = wrap_to_pi(Q_all(candidate_idx, :) - q_ref);
51     dists = vecnorm(diffs, 2, 2);
52     [~, local_idx] = min(dists);
53
54     idx = candidate_idx(local_idx);
55     q_best = Q_all(idx, :);
56 end
57
58
59 function w = wrap_to_pi(x)
60 %WRAP_TO_PI 把角度差折算到 [-pi, pi]。
61     w = mod(x + pi, 2*pi) - pi;
62 end

```

Listing A.8: select_ik_solution.m

A.2 项目三核心代码

A.2.1 路径规划

```

1 function path = plan_path(scene, params)
2 %PLAN_PATH 在给定场景下生成一条由起点到终点的关节空间路径
3 %
4 % path = plan_path(scene, params)
5 %
6 % 输入：
7 %     scene : 场景结构体（见 scene_easy.m / scene_hard.m）
8 %           必须使用的字段：
9 %             scene.q_start 1xn 起始关节角
10 %            scene.q_goal  1xn 目标关节角
11 %            scene.obstacles Mx4 球形障碍 [cx, cy, cz, r]
12 %     params : 由 robot_params() 返回的机械臂参数结构体
13 %
14 % 输出：
15 %     path : Mxn 关节空间路点矩阵
16 %           - 必须满足 path(1, :) == scene.q_start
17 %                 path(end,:) == scene.q_goal
18 %           - 中间行数 M ≥ 2 由所选算法决定

```

```

19 %           - 每个路点须在 params.qlim 内
20 %           - 相邻路点连线在任务空间下不应穿过 scene.obstacles
21 %
22 % 说明：
23 % 本文件为开放性任务。算法可由小组自行选定（直线插值、Bezier、
24 % APF、PRM、RRT 等），实现后请在课程设计报告中说明：
25 %     1. 所选算法名称与适用场景；
26 %     2. 关键参数（步长、迭代上限、采样数 等）；
27 %     3. 在 scene_easy / scene_hard 下的运行结果与失败情形。
28
29 % =====
30 % TODO: 实现你们组的路径规划算法
31 % -----
32 % 要求：
33 %     - 输出 path 必须为 M×n 数值矩阵（n = params.n）
34 %     - 首行为 scene.q_start，末行为 scene.q_goal
35 %     - 中间点须满足 qlim 约束与避障约束
36 %
37 % 未填时返回 NaN 矩阵，下游测试会精确报错指向本 TODO。
38 % =====
39 q_start = scene.q_start(:)';
40 q_goal  = scene.q_goal(:)';
41 n = params.n;
42
43 if numel(q_start) ~= n || numel(q_goal) ~= n
44     error('路径规划失败: scene.q_start / scene.q_goal 维度与 params.n 不一致。');
45 end
46
47 qlim = get_qlim(params, n);
48 q_start = clamp_to_qlim(q_start, qlim);
49 q_goal  = clamp_to_qlim(q_goal, qlim);
50
51 if ~is_state_valid(q_start, scene, params, qlim)
52     error('路径规划失败: 起点构型不合法或与障碍碰撞。');
53 end
54 if ~is_state_valid(q_goal, scene, params, qlim)
55     error('路径规划失败: 终点构型不合法或与障碍碰撞。');
56 end
57
58 edge_samples = 25;
59 if is_edge_valid(q_start, q_goal, scene, params, qlim, edge_samples)
60     path = [q_start; q_goal];
61 else
62     path = plan_with_birrt(q_start, q_goal, scene, params, qlim, edge_samples);
63 end

```

```

64
65     if norm(path(1, :) - q_start) > norm(path(end, :) - q_start)
66         path = flipud(path);
67     end
68     path(1, :) = q_start;
69     path(end, :) = q_goal;
70
71     if any(~isfinite(path(:)))
72         error('路径规划失败: plan_path.m 中的 TODO 尚未完成。');
73     end
74 end
75
76 function path = plan_with_birrt(q_start, q_goal, scene, params, qlim, edge_samples)
77     rng(3, 'twister'); % 固定随机种子, 便于复现实验结果
78
79     n = params.n;
80     max_iter = 6000;
81     step = min(0.35, max(0.12, norm(q_goal - q_start) / 5));
82     connect_tol = step;
83
84     tree_a.q = q_start;
85     tree_a.parent = 0;
86     tree_b.q = q_goal;
87     tree_b.parent = 0;
88
89     for iter = 1:max_iter
90         if mod(iter, 5) == 0
91             q_rand = q_goal;
92         else
93             q_rand = sample_configuration(qlim, q_start, q_goal, iter);
94         end
95
96         [tree_a, idx_new, added] = extend_tree(tree_a, q_rand, step, scene, params,
97 qlim, edge_samples);
98         if added
99             q_new = tree_a.q(idx_new, :);
100             [tree_b, idx_other, reached] = connect_tree(tree_b, q_new, step,
101 scene, params, qlim,
102 edge_samples);
103             if reached
104                 path = merge_trees(tree_a, idx_new, tree_b, idx_other);
105                 path = shortcut_path(path, scene, params, qlim, edge_samples);
106                 return;
107             end
108         end
109     end

```

```

106         end
107
108         tmp = tree_a;
109         tree_a = tree_b;
110         tree_b = tmp;
111     end
112
113     % 确定性候选路点作为兜底，适合本项目两个给定场景。
114     path = try_manual_detours(q_start, q_goal, scene, params, qlim, edge_samples);
115     if isempty(path)
116         error('路径规划失败：RRT 在迭代上限内未找到无碰撞路径。');
117     end
118 end
119
120 function qlim = get_qlim(params, n)
121     if isfield(params, 'qlim') && isequal(size(params.qlim), [n, 2])
122         qlim = params.qlim;
123     else
124         qlim = repmat([-pi, pi], n, 1);
125     end
126 end
127
128 function q = clamp_to_qlim(q, qlim)
129     q = min(max(q, qlim(:, 1)'), qlim(:, 2)');
130 end
131
132 function q = sample_configuration(qlim, q_start, q_goal, iter)
133     n = size(qlim, 1);
134     span = qlim(:, 2)' - qlim(:, 1)';
135
136     if mod(iter, 7) == 0
137         base = (q_start + q_goal) / 2;
138         offsets = zeros(1, n);
139         if n >= 2, offsets(2) = ((-1)^iter) * pi/4; end
140         if n >= 3, offsets(3) = ((-1)^(floor(iter/7))) * pi/3; end
141         if n >= 5, offsets(5) = ((-1)^(floor(iter/11))) * pi/5; end
142         q = base + offsets;
143     else
144         q = qlim(:, 1)' + rand(1, n) .* span;
145     end
146
147     q = clamp_to_qlim(q, qlim);
148 end
149
150 function [tree, idx_new, added] = extend_tree(tree, q_target, step, scene, params,

```

```

qlim, edge_samples)
151   idx_near = nearest_node(tree.q, q_target);
152   q_near = tree.q(idx_near, :);
153   q_new = steer(q_near, q_target, step);
154   q_new = clamp_to_qlim(q_new, qlim);
155
156   added = is_state_valid(q_new, scene, params, qlim) && ...
157           is_edge_valid(q_near, q_new, scene, params, qlim, edge_samples);
158   if added
159       tree.q(end+1, :) = q_new;
160       tree.parent(end+1, 1) = idx_near;
161       idx_new = size(tree.q, 1);
162   else
163       idx_new = idx_near;
164   end
165 end
166
167 function [tree, idx_new, reached] = connect_tree(tree, q_target, step, connect_tol,
scene, params, qlim, edge_samples)
168     reached = false;
169     idx_new = nearest_node(tree.q, q_target);
170
171     while true
172         [tree, idx_new, added] = extend_tree(tree, q_target, step, scene, params,
qlim, edge_samples);
173         if ~added
174             return;
175         end
176         if norm(tree.q(idx_new, :) - q_target) <= connect_tol && ...
177             is_edge_valid(tree.q(idx_new, :), q_target, scene, params, qlim,
edge_samples)
178             tree.q(end+1, :) = q_target;
179             tree.parent(end+1, 1) = idx_new;
180             idx_new = size(tree.q, 1);
181             reached = true;
182             return;
183         end
184     end
185 end
186
187 function q_new = steer(q_from, q_to, step)
188     delta = q_to - q_from;
189     dist = norm(delta);
190     if dist <= step
191         q_new = q_to;

```

```

192     else
193         q_new = q_from + step * delta / dist;
194     end
195 end
196
197 function idx = nearest_node(nodes, q)
198     d = sum((nodes - q).^2, 2);
199     [~, idx] = min(d);
200 end
201
202 function path = merge_trees(tree_a, idx_a, tree_b, idx_b)
203     path_a = trace_path(tree_a, idx_a);
204     path_b = trace_path(tree_b, idx_b);
205     path = [path_a; flipud(path_b)];
206
207     if norm(path(1, :) - tree_b.q(1, :)) < norm(path(1, :) - tree_a.q(1, :))
208         path = flipud(path);
209     end
210 end
211
212 function path = trace_path(tree, idx)
213     path = tree.q(idx, :);
214     while tree.parent(idx) ~= 0
215         idx = tree.parent(idx);
216         path = [tree.q(idx, :); path]; %#ok<AGROW>
217     end
218 end
219
220 function path = shortcut_path(path, scene, params, qlim, edge_samples)
221     i = 1;
222     while i < size(path, 1) - 1
223         shortened = false;
224         for j = size(path, 1):-1:(i+2)
225             if is_edge_valid(path(i, :), path(j, :), scene, params, qlim,
edge_samples)
226                 path = [path(1:i, :); path(j:end, :)];
227                 shortened = true;
228                 break;
229             end
230         end
231         if ~shortened
232             i = i + 1;
233         end
234     end
235 end

```

```

236
237 function path = try_manual_detours(q_start, q_goal, scene, params, qlim,
    edge_samples)
238     n = params.n;
239     base = (q_start + q_goal) / 2;
240     candidates = {};
241     magnitudes = [pi/6, pi/4, pi/3, pi/2];
242     joints = unique([2, 3, 5, min(n, 1)]);
243
244     for mag = magnitudes
245         for s2 = [-1, 1]
246             q_mid = base;
247             for j = joints
248                 q_mid(j) = q_mid(j) + s2 * mag;
249             end
250             candidates{end+1} = [q_start; clamp_to_qlim(q_mid, qlim); q_goal]; %#ok<
AGROW>
251         end
252         if n >= 3
253             q1 = base; q2 = base;
254             q1(2) = q1(2) + mag; q1(3) = q1(3) - mag;
255             q2(2) = q2(2) - mag; q2(3) = q2(3) + mag;
256             candidates{end+1} = [q_start; clamp_to_qlim(q1, qlim); clamp_to_qlim(q2,
qlim); q_goal]; %#ok<AGROW>
257             candidates{end+1} = [q_start; clamp_to_qlim(q2, qlim); clamp_to_qlim(q1,
qlim); q_goal]; %#ok<AGROW>
258         end
259     end
260
261     path = [];
262     for i = 1:numel(candidates)
263         p = candidates{i};
264         ok = true;
265         for k = 1:size(p, 1)-1
266             if ~is_edge_valid(p(k, :), p(k+1, :), scene, params, qlim, edge_samples)
267                 ok = false;
268                 break;
269             end
270         end
271         if ok
272             path = p;
273             return;
274         end
275     end
276 end

```



```

277
278 function ok = is_state_valid(q, scene, params, qlim)
279     ok = all(isfinite(q)) && all(q >= qlim(:, 1)' - 1e-9) && all(q <= qlim(:, 2)' +
280         1e-9);
281     if ok
282         ok = ~is_collision(q, scene, params);
283     end
284
285 function ok = is_edge_valid(q1, q2, scene, params, qlim, edge_samples)
286     ok = true;
287     for s = linspace(0, 1, edge_samples)
288         q = (1 - s) * q1 + s * q2;
289         if ~is_state_valid(q, scene, params, qlim)
290             ok = false;
291             return;
292         end
293     end
294 end
295
296 function collision = is_collision(q, scene, params)
297     collision = false;
298     if ~isfield(scene, 'obstacles') || isempty(scene.obstacles)
299         return;
300     end
301
302     [~, T_all] = forward_kinematics(q, params);
303     pts = squeeze(T_all(1:3, 4, :))';
304
305     for k = 1:size(scene.obstacles, 1)
306         center = scene.obstacles(k, 1:3);
307         radius = scene.obstacles(k, 4);
308         for j = 1:size(pts, 1)-1
309             if seg_point_dist(pts(j, :), pts(j+1, :), center) < radius
310                 collision = true;
311                 return;
312             end
313         end
314     end
315 end
316
317 function d = seg_point_dist(A, B, C)
318     AB = B - A;
319     L2 = AB * AB';
320     if L2 < 1e-12

```

```

321     d = norm(A - C);
322     return;
323 end
324 t = max(0, min(1, dot(C - A, AB) / L2));
325 P = A + t * AB;
326 d = norm(P - C);
327 end

```

Listing A.9: plan_path.m

A.2.2 轨迹生成

```

1  function traj = generate_trajectory(path, scene, params)
2  %GENERATE_TRAJECTORY 把离散关节路点 path 扩展成时间参数化的连续轨迹
3  %
4  %   traj = generate_trajectory(path, scene, params)
5  %
6  %   输入：
7  %       path    : M×n 关节路点矩阵（来自 plan_path）
8  %       scene   : 场景结构体，使用其 T_total 与 dt 字段
9  %       params  : 机械臂参数结构体
10 %
11 %   输出：
12 %       traj    : 轨迹结构体，字段约定如下
13 %           traj.t    1×K 时间序列，t(1)=0、t(end)=scene.T_total
14 %           traj.q    K×n 关节位置序列，首行 == path(1,:)、末行 == path(end,:)
15 %           traj.qd   K×n 关节速度序列，首末行均为 0
16 %           traj.qdd  K×n 关节加速度序列，首末行均为 0
17 %           K = round(T_total/dt) + 1
18 %
19 %   说明：
20 %   本文件为开放性任务。插值方法可由小组自行选定（线性、三次多项式、
21 %   五次多项式、梯形速度规划、S 形规划 等），实现后请在课程设计报告中说明：
22 %       1. 所选插值方法；
23 %       2. 边界条件（首末速度 / 加速度的设置方式）；
24 %       3. 关节速度 / 加速度峰值与机械臂可行范围的对比。
25 %
26 %   =====
27 %   TODO: 实现你们组的轨迹生成
28 %   -----
29 %   要求：
30 %       - traj.t、traj.q、traj.qd、traj.qdd 字段维度满足上述约定
31 %       - traj.q 在首末点与 path 一致
32 %       - traj.qd、traj.qdd 在首末时刻为 0

```

```

33     % - 离散步长 dt 取 scene.dt
34     %
35     % 未填时各字段为 NaN，下游测试会精确报错指向本 TODO。
36     % =====
37     K = round(scene.T_total / scene.dt) + 1;
38     n = params.n;
39
40     if size(path, 2) ~= n || size(path, 1) < 2
41         error('轨迹生成失败: path 维度应为 M×params.n, 且 M >= 2。');
42     end
43
44     traj.t = linspace(0, scene.T_total, K);
45
46     % 用五次多项式时间缩放 s(t)，沿 path 的关节空间弧长前进。
47     tau = traj.t / scene.T_total;
48     s = 10*tau.^3 - 15*tau.^4 + 6*tau.^5;
49
50     ds_dt = (30*tau.^2 - 60*tau.^3 + 30*tau.^4) / scene.T_total;
51     d2s_dt2 = (60*tau - 180*tau.^2 + 120*tau.^3) / (scene.T_total^2);
52
53     seg_vec = diff(path, 1, 1);
54     seg_len = sqrt(sum(seg_vec.^2, 2));
55     keep = [true; seg_len > 1e-12];
56     path = path(keep, :);
57     seg_vec = diff(path, 1, 1);
58     seg_len = sqrt(sum(seg_vec.^2, 2));
59
60     if isempty(seg_len) || sum(seg_len) < 1e-12
61         traj.q = repmat(path(1, :), K, 1);
62         traj.qd = zeros(K, n);
63         traj.qdd = zeros(K, n);
64     else
65         u_node = [0; cumsum(seg_len) / sum(seg_len)];
66         u = s(:);
67
68         traj.q = zeros(K, n);
69         traj.qd = zeros(K, n);
70         traj.qdd = zeros(K, n);
71
72         for k = 1:K
73             if u(k) >= 1
74                 idx = numel(seg_len);
75                 local = 1;
76             else
77                 idx = find(u(k) >= u_node(1:end-1) & u(k) <= u_node(2:end), 1, '

```

```

first');
78         if isempty(idx)
79             idx = 1;
80         end
81         denom = u_node(idx+1) - u_node(idx);
82         local = (u(k) - u_node(idx)) / denom;
83     end
84
85     dq_du = seg_vec(idx, :) / (u_node(idx+1) - u_node(idx));
86     traj.q(k, :) = path(idx, :) + local * seg_vec(idx, :);
87     traj.qd(k, :) = dq_du * ds_dt(k);
88     traj.qdd(k, :) = dq_du * d2s_dt2(k);
89 end
90
91 traj.q(1, :) = path(1, :);
92 traj.q(end, :) = path(end, :);
93 traj.qd(1, :) = 0;
94 traj.qd(end, :) = 0;
95 traj.qdd(1, :) = 0;
96 traj.qdd(end, :) = 0;
97 end
98
99 if any(~isfinite(traj.t)) || any(~isfinite(traj.q(:))) ...
100    || any(~isfinite(traj.qd(:))) || any(~isfinite(traj.qdd(:)))
101     error('轨迹生成失败: generate_trajectory.m 中的 TODO 尚未完成。');
102 end
103 end

```

Listing A.10: generate_trajectory.m

A.2.3 PD 控制器

```

1 function tau = pd_controller(q, qd, q_des, qd_des, qdd_ref, params, gains)
2 %PD_CONTROLLER 关节空间 PD 控制器 (含前馈加速度)
3 %
4 % tau = pd_controller(q, qd, q_des, qd_des, qdd_ref, params, gains)
5 %
6 % 控制律:
7 % tau = Kp .* (q_des - q) + Kd .* (qd_des - qd) + qdd_ref
8 %
9 % 输入:
10 % q : 1xn 当前关节位置
11 % qd : 1xn 当前关节速度
12 % q_des : 1xn 期望关节位置

```

```

13 %      qd_des   : 1xn 期望关节速度
14 %      qdd_ref  : 1xn 前馈加速度 (可为零向量)
15 %      params   : 机械臂参数结构体 (保留接口, 本控制器未使用动力学参数)
16 %      gains    : struct, 包含
17 %                  gains.Kp  1xn 比例增益
18 %                  gains.Kd  1xn 微分增益
19 %
20 %  输出:
21 %      tau       : 1xn 关节控制量 (在 simulate_tracking 的简化等效惯性
22 %                  假设下等价于关节加速度指令)
23 %
24 %  说明:
25 %  本课程不要求建模完整动力学, 因此 tau 在 simulate_tracking 内部
26 %  按"等效惯性矩阵 = I"折算为关节加速度。如需要扩展到含重力补偿的
27 %  计算力矩控制, 可在本函数末尾追加 G(q) 项。
28
29 q       = q(:)';
30 qd      = qd(:)';
31 q_des   = q_des(:)';
32 qd_des  = qd_des(:)';
33 qdd_ref = qdd_ref(:)';
34
35 Kp = gains.Kp(:)';
36 Kd = gains.Kd(:)';
37
38 tau = Kp .* (q_des - q) + Kd .* (qd_des - qd) + qdd_ref;
39 end

```

Listing A.11: pd_controller.m

A.2.4 跟踪仿真

```

1 function result = simulate_tracking(traj, scene, params, mode)
2 %SIMULATE_TRACKING 跟踪仿真主循环 (运动学 / 简化动力学双模式)
3 %
4 %      result = simulate_tracking(traj, scene, params, mode)
5 %
6 %  输入:
7 %      traj    : 由 generate_trajectory 返回的轨迹结构体
8 %      scene   : 场景结构体 (用于绘制障碍)
9 %      params  : 机械臂参数结构体
10 %      mode    : 'kinematic' (默认) 或 'dynamic'
11 %               - 'kinematic': 直接把 traj.q 作为实际关节序列
12 %               - 'dynamic' : 用 pd_controller 闭环跟踪 traj,

```

```

13 %                                在简化等效惯性  $M_{eff} = I$  下用前向 Euler 积分
14 %
15 % 输出：
16 %     result : 结构体，字段如下
17 %         result.t        1×K 时间序列
18 %         result.q        K×n 实际关节序列
19 %         result.q_des    K×n 期望关节序列（即 traj.q）
20 %         result.err_pos  K×1 末端位置跟踪误差（基坐标系，欧氏距离）
21 %         result.err_q    K×n 关节角误差（ $q_{des} - q$ ）
22 %         result.mode     'kinematic' 或 'dynamic'
23 %
24 % 说明：
25 % 本函数为老师提供的完整实现。'dynamic' 模式下假定等效惯性矩阵
26 % 为单位阵、忽略科氏 / 重力 / 摩擦，便于学生在不依赖 RTB
27 % 动力学参数（质量、惯量张量）的条件下观察 PD 增益的影响。
28 % 动画通过 RTB 的 SerialLink.plot 实现，每隔 plot_every 帧刷新一次。
29
30 if nargin < 4 || isempty(mode)
31     mode = 'kinematic';
32 end
33
34 K = numel(traj.t);
35 n = params.n;
36
37 result.t      = traj.t;
38 result.q_des  = traj.q;
39 result.q      = zeros(K, n);
40 result.err_pos = zeros(K, 1);
41 result.err_q   = zeros(K, n);
42 result.mode    = mode;
43
44 % -----
45 % 主循环
46 % -----
47 switch lower(mode)
48     case 'kinematic'
49         % 运动学模式:  $q(t)$  直接来自轨迹（理想跟踪）
50         result.q = traj.q;
51
52     case 'dynamic'
53         % 简化动力学模式:  $q'' = \tau$  ( $M_{eff} = I$ )
54         % 默认 PD 增益（关节空间对角）
55         gains.Kp = 100 * ones(1, n);
56         gains.Kd = 20 * ones(1, n);
57

```

```

58     q = traj.q(1, :);
59     qd = zeros(1, n);
60
61     result.q(1, :) = q;
62
63     for k = 1:K-1
64         dt = traj.t(k+1) - traj.t(k);
65
66         tau = pd_controller(q, qd, ...
67                             traj.q(k, :), traj.qd(k, :), ...
68                             traj.qdd(k, :), params, gains);
69
70         %  $M_{eff} = I \rightarrow qdd = tau$ 
71         qdd = tau;
72
73         % 前向 Euler 积分
74         qd = qd + qdd * dt;
75         q = q + qd * dt;
76
77         % 关节限位裁剪
78         if isfield(params, 'qlim') && ~isempty(params.qlim)
79             q = min(max(q, params.qlim(:, 1)'), params.qlim(:, 2)');
80         end
81
82         result.q(k+1, :) = q;
83     end
84
85     otherwise
86         error('未知的仿真模式 '%s'', 可选 'kinematic' 或 'dynamic'。', mode);
87     end
88
89     % -----
90     % 误差统计
91     % -----
92     result.err_q = result.q_des - result.q;
93
94     for k = 1:K
95         T_des = forward_kinematics(result.q_des(k, :), params);
96         T_act = forward_kinematics(result.q(k, :), params);
97         result.err_pos(k) = norm(T_des(1:3, 4) - T_act(1:3, 4));
98     end
99
100    % -----
101    % RTB 动画
102    % -----

```

```

103     try
104         robot = build_rtb_robot(params);
105     catch ME
106         warning('build_rtb_robot 调用失败: %s\n跳过动画。', ME.message);
107         return;
108     end
109
110     % ----- 先算自适应 workspace -----
111     % 取轨迹所有 frame 原点 + 障碍球外接盒 + base, 外扩 padding
112     all_pts = zeros(K * (params.n + 1), 3);
113     for k = 1:K
114         [~, T_all_k] = forward_kinematics(result.q(k, :), params);
115         rows = (k - 1) * (params.n + 1) + (1 : params.n + 1);
116         all_pts(rows, :) = squeeze(T_all_k(1:3, 4, :));
117     end
118     all_pts = [all_pts; 0, 0, 0]; % base
119     if isfield(scene, 'obstacles') && ~isempty(scene.obstacles)
120         obs_lo = scene.obstacles(:, 1:3) - scene.obstacles(:, 4);
121         obs_hi = scene.obstacles(:, 1:3) + scene.obstacles(:, 4);
122         all_pts = [all_pts; obs_lo; obs_hi];
123     end
124     pad = 100; % mm
125     ws_min = min(all_pts, [], 1) - pad;
126     ws_max = max(all_pts, [], 1) + pad;
127     workspace = [ws_min(1), ws_max(1), ws_min(2), ws_max(2), ws_min(3), ws_max(3)];
128
129     figure('Name', sprintf('Project 3 跟踪仿真 - %s / %s', params.name, mode));
130     hold on;
131
132     % 1) 自绘末端轨迹 (独立于 RTB trail, 一次画完整条蓝线)
133     ee_pos = zeros(K, 3);
134     for k = 1:K
135         T_k = forward_kinematics(result.q(k, :), params);
136         ee_pos(k, :) = T_k(1:3, 4)';
137     end
138     plot3(ee_pos(:, 1), ee_pos(:, 2), ee_pos(:, 3), 'b-', 'LineWidth', 1.5);
139
140     % 2) 障碍球
141     if isfield(scene, 'obstacles') && ~isempty(scene.obstacles)
142         [Xs, Ys, Zs] = sphere(20);
143         for i = 1:size(scene.obstacles, 1)
144             cx = scene.obstacles(i, 1);
145             cy = scene.obstacles(i, 2);
146             cz = scene.obstacles(i, 3);
147             r = scene.obstacles(i, 4);

```



```
148         surf(r*Xs + cx, r*Ys + cy, r*Zs + cz, ...
149             'FaceAlpha', 0.3, 'EdgeColor', 'none', 'FaceColor', [1 0.4 0.4]);
150     end
151 end
152
153 % 3) 锁定 axis 到 workspace (RTB plot 调用前后都重锁一次)
154 xlim(workspace(1:2)); ylim(workspace(3:4)); zlim(workspace(5:6));
155 set(gca, 'XLimMode', 'manual', 'YLimMode', 'manual', 'ZLimMode', 'manual');
156 daspect([1 1 1]);
157
158 % 4) RTB 动画 (不带 'trail', 蓝线已在第 1 步画好)
159 plot_every = max(1, round(K / 60));
160 idx_plot = unique([1 : plot_every : K, K]);
161 robot.plot(result.q(idx_plot, :), 'fps', 20, ...
162     'noshadow', 'workspace', workspace);
163
164 % 5) 动画结束后再 reset, 防止 plot 内部把 axis 改掉
165 xlim(workspace(1:2)); ylim(workspace(3:4)); zlim(workspace(5:6));
166 set(gca, 'XLimMode', 'manual', 'YLimMode', 'manual', 'ZLimMode', 'manual');
167 hold off;
168 end
```

Listing A.12: simulate_tracking.m

附录 B 运行命令与提交前检查表

B.1 MATLAB 运行方式

```
1 addpath('../project1');
2 addpath('../project2');
3 addpath('../project3');
4
5 cd('../project2');
6 main_project2_test;
7
8 cd('../project3');
9 main_project3_test;
```

B.2 LaTeX 编译方式

在 latex_detailed_report 文件夹中使用 XeLaTeX:

```
1 xelatex main.tex
2 xelatex main.tex
```

编译两次是为了让目录、页码和交叉引用稳定。

B.3 提交前检查表

1. 封面小组成员、学号、教师和日期已填写;
2. 文档中“数值 IK 与解析 IK 的区别”表述真实;
3. easy、hard 场景均已实际运行;
4. 测试输出来自本机真实运行, 没有手工伪造;
5. 3D 场景图和 $q, \dot{q}, \ddot{q}$ 曲线已加入;
6. 最小安全距离曲线已加入;
7. 速度、加速度峰值与允许值做了比较;
8. AI 工具使用声明按实际情况修改;

9. 每位成员能解释 DLS、碰撞检测、BiRRT 和五次时间缩放；
10. 最终 PDF 中目录、公式、中文、代码换行均正常。

结语

项目二解决“给定关节角，末端在哪里”和“给定末端位姿，关节应该转多少”；项目三解决“从一个构型到另一个构型，怎样避开障碍并平稳地走过去”。把这两部分连起来，就得到一条完整的机器人运动链：

建模 → 正/逆运动学 → 碰撞检测 → 路径规划 → 轨迹生成 → 闭环跟踪.

真正需要掌握的不是背下某一行 MATLAB，而是知道每一行代码对应上面链条中的哪一个物理问题、数学问题和工程约束。